

Guida tecnica completa

Firmware Tank + controller ESP32

Lettura per blocchi, baseline e aggiornamento post-correzioni

Baseline della guida: 7f43f59c00b3c0022841e7f037ebe5a3e8a88235
Snapshot analizzata: working tree post-correzioni e refactor cingoli, 19 luglio 2026
Firmware coperto: tank/ e controller-esp32/
Sorgenti esclusi: CAD, librerie vendor e file README generici
Documento collegato: audit_firmware_tank_esp32.pdf

I listing vengono caricati direttamente dai sorgenti e quindi mostrano il firmware corrente alla compilazione del PDF. Le sezioni dei cingoli e della lettura joystick sono state riallineate riga per riga alla snapshot del 19 luglio 2026; per gli altri moduli il capitolo “Aggiornamento post-correzioni” descrive il delta rispetto alla baseline. I comportamenti hardware restano distinti da quelli dimostrabili dal software.

Indice

1	Come leggere la guida	4
2	Architettura del sistema	4
3	Tecnologie e concetti trasversali	4
3.1	PlatformIO, Arduino e C++	4
3.2	WiFi AP, station e UDP	4
3.3	ADC, INPUT_PULLUP e joystick	5
3.4	I2C, PCA9685 e PWM	5
3.5	Motori, servo, stepper e relay	5
3.6	millis, polling e interrupt	5
4	Contratto UDP e verifica incrociata	5
5	Aggiornamento post-correzioni: cosa e' stato modificato	6
5.1	Come usare questo capitolo	6
5.2	Mappa delle modifiche	6
5.3	Flusso di sicurezza attuale	7
5.4	Receiver UDP del tank: delta tecnico	7
5.5	Main tank, Fire/Zero e fault PCA	7
5.6	PCA, motori e inversione	7
5.7	Controller: calibrazione, pulsanti e socket UDP	8
6	Lettura completa dei file: baseline e sorgente corrente	8
6.1	Configurazione PlatformIO del tank	8
6.1.1	Panoramica	8
6.1.2	Blocchi logici	8
6.1.3	Funzioni e failure mode	8
6.1.4	Listing completo numerato	9
6.1.5	Lettura riga per riga	9
6.2	Coordinatore principale del tank	10
6.2.1	Panoramica	10
6.2.2	Blocchi logici	10
6.2.3	Funzioni e failure mode	10
6.2.4	Listing completo numerato	10
6.2.5	Lettura riga per riga	14
6.3	Interfaccia del ricevitore UDP	18
6.3.1	Panoramica	18
6.3.2	Blocchi logici	18
6.3.3	Funzioni e failure mode	18
6.3.4	Listing completo numerato	18
6.3.5	Lettura riga per riga	19
6.4	AP WiFi e parser del comando UDP	20
6.4.1	Panoramica	20
6.4.2	Blocchi logici	20
6.4.3	Funzioni e failure mode	20
6.4.4	Listing completo numerato	20
6.4.5	Lettura riga per riga	23
6.5	Interfaccia e configurazione dei cingoli	26
6.5.1	Panoramica	26
6.5.2	Blocchi logici	26
6.5.3	Funzioni e failure mode	26
6.5.4	Listing completo numerato	26

6.5.5	Lettura riga per riga	27
6.6	Mixing differenziale e PWM dei cingoli	29
6.6.1	Panoramica	29
6.6.2	Blocchi logici	29
6.6.3	Funzioni e failure mode	29
6.6.4	Listing completo numerato	29
6.6.5	Lettura riga per riga	32
6.7	Interfaccia torretta ed elevazione	34
6.7.1	Panoramica	34
6.7.2	Blocchi logici	34
6.7.3	Funzioni e failure mode	34
6.7.4	Listing completo numerato	34
6.7.5	Lettura riga per riga	35
6.8	Sequenza stepper e controllo dei servo	37
6.8.1	Panoramica	37
6.8.2	Blocchi logici	37
6.8.3	Funzioni e failure mode	37
6.8.4	Listing completo numerato	37
6.8.5	Lettura riga per riga	40
6.9	Interfaccia semantica della shield motori	44
6.9.1	Panoramica	44
6.9.2	Blocchi logici	44
6.9.3	Funzioni e failure mode	44
6.9.4	Listing completo numerato	44
6.9.5	Lettura riga per riga	45
6.10	Traduzione motori e servo in canali PCA	47
6.10.1	Panoramica	47
6.10.2	Blocchi logici	47
6.10.3	Funzioni e failure mode	47
6.10.4	Listing completo numerato	47
6.10.5	Lettura riga per riga	49
6.11	Interfaccia minima PCA9685	52
6.11.1	Panoramica	52
6.11.2	Blocchi logici	52
6.11.3	Funzioni e failure mode	52
6.11.4	Listing completo numerato	52
6.11.5	Lettura riga per riga	53
6.12	Scritture I2C verso PCA9685	54
6.12.1	Panoramica	54
6.12.2	Blocchi logici	54
6.12.3	Funzioni e failure mode	54
6.12.4	Listing completo numerato	54
6.12.5	Lettura riga per riga	55
6.13	Configurazione PlatformIO del controller	58
6.13.1	Panoramica	58
6.13.2	Blocchi logici	58
6.13.3	Funzioni e failure mode	58
6.13.4	Listing completo numerato	58
6.13.5	Lettura riga per riga	58
6.14	Scheduler del controller ESP32	59
6.14.1	Panoramica	59
6.14.2	Blocchi logici	59
6.14.3	Funzioni e failure mode	59
6.14.4	Listing completo numerato	59
6.14.5	Lettura riga per riga	60

6.15	Contratto dei dati del controller	61
6.15.1	Panoramica	61
6.15.2	Blocchi logici	61
6.15.3	Funzioni e failure mode	61
6.15.4	Listing completo numerato	61
6.15.5	Lettura riga per riga	62
6.16	ADC, calibrazione e pulsanti ESP32	63
6.16.1	Panoramica	63
6.16.2	Blocchi logici	63
6.16.3	Funzioni e failure mode	63
6.16.4	Listing completo numerato	63
6.16.5	Lettura riga per riga	67
6.17	Interfaccia del sender UDP	71
6.17.1	Panoramica	71
6.17.2	Blocchi logici	71
6.17.3	Funzioni e failure mode	71
6.17.4	Listing completo numerato	71
6.17.5	Lettura riga per riga	71
6.18	Connessione station e serializzazione UDP	73
6.18.1	Panoramica	73
6.18.2	Blocchi logici	73
6.18.3	Funzioni e failure mode	73
6.18.4	Listing completo numerato	73
6.18.5	Lettura riga per riga	76
7	Riepilogo di studio e limiti pratici	78

1 Come leggere la guida

Ogni file e' presentato con quattro livelli complementari: una panoramica, la tabella dei blocchi logici, un listing completo con numeri di riga e una tabella riga per riga. Il listing e' la fonte principale per vedere sintassi e commenti. Per `trackController.*` e `joystickReader.*`, tabelle e listing sono allineati alla snapshot corrente; per gli altri moduli modificati usare prima il capitolo "Aggiornamento post-correzioni", poi il listing corrente e infine la spiegazione baseline delle righe non cambiate.

I riferimenti A-01 fino a A-14 rimandano agli ID dell'audit allegato. Lo stato aggiornato (risolto, mitigato o aperto) e' nella tabella post-correzioni dell'audit; un riferimento non significa che la singola riga sia ancora errata.

2 Architettura del sistema

ESP32 controller

```

ADC joystick + pulsanti
-> WiFi station, UDP V1 / porta 4210
-> Uno R4 WiFi: AP Tank_AP + UdpReceiver_update()
-> TankCommand
    -> TrackController -> MotorController -> PWMController -> I2C/PCA9685 -> M3/M1
    -> ServoTorretta -> MotorController -> PCA9685 -> S5/S6
    -> StepperTorretta -> D2-D5 -> driver stepper
    -> Relay Fire -> D7

```

Segnale	Origine/destinazione	Significato
driveX	GPIO32 dopo lo swap -> tank	Sterzata differenziale.
driveY	GPIO34 dopo lo swap -> tank	Avanti/indietro.
turretX	GPIO33 dopo lo swap -> D2-D5	Rotazione orizzontale.
elevationY	GPIO35 dopo lo swap -> S5/S6	Elevazione servo.
zero, fire	GPIO25/GPIO26, LOW attivo	Azzeramento e impulso relay.
I2C PCA9685	Uno R4 -> shield	PWM motori M1/M3 e servo S5/S6.

3 Tecnologie e concetti trasversali

3.1 PlatformIO, Arduino e C++

PlatformIO legge `platformio.ini`, risolve framework/librerie e compila. Il modello Arduino chiama `setup()` una volta e poi `loop()` ripetutamente. Gli header `.h` dichiarano tipi e API; i `.cpp` definiscono il comportamento. Le include guard impediscono doppie dichiarazioni. Macro `#define` sono sostituzioni del preprocessore: non hanno tipo e vanno usate con attenzione.

3.2 WiFi AP, station e UDP

Il tank crea l'access point; l'ESP32 e' una station. UDP invia datagrammi a una porta senza handshake applicativo: non garantisce consegna, ordine, unicit , autenticita' o conferma. Il frame V1 e' `V1;driveX;driveY;turretX;elevationY;zero;fire`. Un frame normale massimo e' 26 byte; il buffer TX da 64 byte e' adeguato. Su richiesta, il receiver conserva il parsing compatibile con la baseline: legge al massimo 63 byte, usa `sscanf` per sei interi e limita gli assi con `constrain`. Non rifiuta rigorosamente suffissi, campi extra o conversioni `int` fuori range; V1 inoltre non autentica mittente, sequenza, replay o freschezza.

`WL_CONNECTED` e `udpReady` provano solo lo stato locale della station/socket, non che il tank abbia applicato il comando.

3.3 ADC, INPUT_PULLUP e joystick

L'ESP32 legge tensioni analogiche con ADC. La risoluzione 12 bit produce normalmente 0–4095, poi `map` le adatta a 0–1023. `constrain` limita il valore. GPIO32–35 sono ADC1: questa scelta evita il conflitto WiFi tipico di ADC2. Con `INPUT_PULLUP`, il pin e' HIGH a riposo e LOW alla pressione. Il controller applica una deadzone di ingresso da 20 alla guida e da 80 alla torretta: filtrano il rumore del potenziometro attorno al centro calibrato. Il tank conserva una deadzone distinta da 100 sul comando cingolo, prima e dopo il mixing, per proteggere anche da sender UDP diversi dall'ESP32. Il firmware valida centro/spread alla calibrazione, applica debounce di 40 ms e richiede neutralita' prima dell'armamento; non puo' pero' diagnosticare ogni guasto di potenziometro o cablaggio dopo il boot.

3.4 I2C, PCA9685 e PWM

I2C e' un bus indirizzato: il master invia indirizzo, registro e byte, mentre un ACK indica soltanto che un dispositivo ha risposto. La PCA9685 offre 16 canali PWM a 12 bit; i registri ON/OFF descrivono il ciclo. Il valore 4096 viene usato come flag full-on/full-off. Tutti i canali condividono la stessa frequenza: qui 50 Hz e' corretto per servo ma un compromesso per motori DC. Il firmware ora propaga NACK, letture corte e canali non validi in un fault latched; una chiamata I2C gia' bloccata e l'ultimo PWM fisico richiedono comunque un enable hardware indipendente.

3.5 Motori, servo, stepper e relay

Un H-bridge decide verso e PWM dei motori DC. Il firmware ora porta PWM a zero prima di cambiare verso e il controller cingoli introduce un dead-time di 30 ms; non e' una rampa completa. Il servo interpreta la larghezza di un impulso periodico; la coppia S5/S6 usa un angolo specchiato. Lo stepper e' pilotato da quattro fasi D2–D5, senza feedback di posizione. Il relay traduce D7 in contatto elettrico: durata di impulso, polarita' e stato al reset sono requisiti safety e non solo dettagli software.

3.6 millis, polling e interrupt

Il progetto non definisce ISR o `attachInterrupt`. Tutto avviene per polling nel loop. Il pattern `millis()-last>=interval` e' corretto attraverso il rollover unsigned, ma una scadenza e' verificata solo se il loop torna a eseguire. WiFi, I2C, Serial e `delay` possono introdurre jitter. Nessuna ISR applicativa significa nessuna race `volatile` nel codice visibile, non significa real-time garantito.

4 Contratto UDP e verifica incrociata

Campo	Controller	Tank
<code>driveX</code>	GPIO32 dopo swap, 0–1023	Sterzo differenziale.
<code>driveY</code>	GPIO34 dopo swap, 0–1023	Avanti/indietro.
<code>turretX</code>	GPIO33 dopo swap, 0–1023	Stepper torretta.
<code>elevationY</code>	GPIO35 dopo swap, 0–1023	Servo elevazione.
<code>zero</code>	GPIO25, LOW = 1	Fronte Zero.
<code>fire</code>	GPIO26, LOW = 1	Fronte relay Fire.

Il sender serializza i sei campi nell'ordine mostrato e il receiver usa lo stesso ordine. Questa coerenza e' un aspetto positivo. A-08 resta aperto per scelta di compatibilita' del parser `sscanf`; restano inoltre A-04 e A-09, cioe' autenticazione, anti-replay, ordine, freschezza e conferma applicativa.

5 Aggiornamento post-correzioni: cosa e' stato modificato

5.1 Come usare questo capitolo

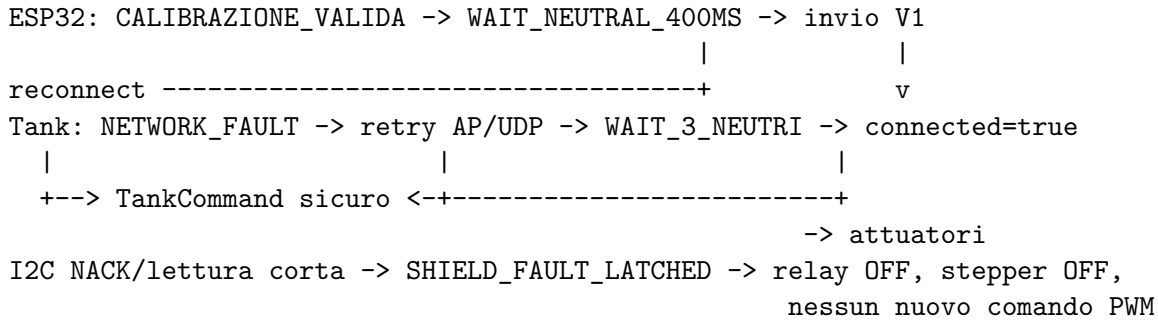
Questa e' la revisione della guida richiesta dopo l'implementazione dei fix. Il protocollo UDP V1, la porta 4210 e i pin fisici non sono cambiati. Sono invece cambiati undici moduli del firmware e alcune API locali: `PWMController::begin` e `MotorController::begin` ora restituiscono un esito, `MotorController` espone lo stato I2C e `JoystickReader` espone il riarmo neutro.

I listing sotto "Lettura completa dei file" usano `\textbackslashstinputlisting` e sono quindi aggiornati automaticamente al sorgente corrente quando questo PDF viene compilato. Le tabelle riga per riga della baseline restano utili per gli elementi invariati; per ogni modulo nella tabella seguente, questa sezione prevale su vecchi numeri di riga, vecchie panoramiche e frasi che descrivono il comportamento prima dei fix.

5.2 Mappa delle modifiche

File	Cosa e' stato cambiato	Effetto e limite
<code>tank/src/udpReceiver.cpp</code>	Stato <code>NETWORK_FAULT/NETWORK_READY</code> , retry AP/UDP, parser V1 <code>sscanf</code> compatibile e riarmo neutro.	Non esiste piu' <code>while(true)</code> ; il parser accetta sei interi e clampa gli assi, quindi A-08 resta aperto. Una chiamata Wi-FiS3 puo' comunque bloccarsi internamente.
<code>tank/src/udpReceiver.h</code>	<code>TankCommand.connected</code> documenta ora "comando autorizzato dal riarmo", non soltanto traffico recente.	Il main deve trattare <code>false</code> come divieto assoluto di comandare attuatori.
<code>tank/src/main.cpp</code>	GPIO sicuri prima di Serial, guardia globale su link/shield, Fire e Zero con rilascio obbligatorio, fault I2C latched.	Spegne i segnali controllabili dal firmware; non sostituisce un enable elettrico esterno.
<code>tank/lib/PWMController/*</code>	Le primitive I2C restituiscono <code>bool</code> e memorizzano <code>communicationHealthy</code> .	Rileva NACK, letture corte e canali invalidi quando la chiamata ritorna; non sblocca Wire.
<code>tank/lib/motorController/*</code>	Inizializzazione e health I2C propagati; PWM zero prima del cambio direzione.	Evita la commutazione software con PWM attivo, ma non e' una rampa completa.
<code>tank/src/trackController.*</code>	Deadzone di sicurezza <code>TRACK_COMMAND_DEADZONE=100</code> prima/dopo mixing, stato M1/M3 e dead-time di 30 ms durante inversione.	Lo stato conserva verso e istante dell'ultimo stop anche dopo il ritorno al centro: una richiesta immediata del verso opposto non salta la pausa. Valori e frenatura fisica restano da misurare sul veicolo.
<code>controller-esp32/src/joystickReader.*</code>	Deadzone di ingresso separate (guida 20, torretta 80), calibrazione validata, debounce e riarmo neutro di 400 ms.	Le inversioni di Joy1 sono applicate solo qui; se la calibrazione fallisce il controller invia solo dati sicuri fino al riavvio.
<code>controller-esp32/src/udpSender.cpp</code>	Stato <code>udpReady</code> , controllo delle API UDP e soppressione del campione pre-reconnect.	Conferma soltanto lo stato locale del socket, non la ricezione/applicazione sul tank.

5.3 Flusso di sicurezza attuale



Il doppio riarmo e' intenzionale: il controller non trasmette subito un comando attivo dopo boot/reconnect e il tank non si fida comunque del primo datagramma che `sscanf` riesce a interpretare. Nessuna freccia del diagramma dimostra il taglio fisico dell'alimentazione: quello richiede hardware dedicato.

5.4 Receiver UDP del tank: delta tecnico

- `UdpReceiver_begin()` non avvia piu' AP e socket in modo definitivo nel `setup`. Azzera lo stato, lascia il comando sicuro e programma il primo tentativo dal `loop`.
- `ensureNetworkReady()` tenta `WiFi.beginAP()` e `udp.begin()` al massimo una volta ogni 3 s dopo un errore restituito. Il timestamp viene aggiornato dopo la chiamata, per evitare retry immediati se la libreria ha atteso a lungo.
- Il parser e' stato riportato alla semantica originale: `sscanf("V1;%d;...", ...)` deve convertire sei interi; gli assi sono poi limitati con `constrain(..., 0, 1023)` e i pulsanti valgono true solo se uguali a 1.
- La lettura copia al massimo 63 byte e aggiunge il NUL, ma non rifiuta esplicitamente payload oversize, suffissi, campi extra o conversioni `int` fuori range. Se `sscanf` non converte sei campi il watchdog non viene aggiornato; un datagramma troncato che contiene gia' sei conversioni puo' invece essere accettato.
- `controlsArmed` diventa vero solo dopo tre frame validi con assi entro 512 ± 20 e Fire/Zero rilasciati. Prima di allora `connected` resta falso e il main ferma tutto.

Resta un datagramma elaborato per chiamata; V1 non contiene sequence number, lease, HMAC o anti-replay. In particolare, "retry controllato" non vuol dire che `WiFi.beginAP()` o `udp.parsePacket()` siano non bloccanti.

5.5 Main tank, Fire/Zero e fault PCA

- Il latch inattivo di D7 e i latch LOW di D2–D5 vengono scritti prima di `Serial.begin()` e `delay(1000)`. Questo riduce la finestra software di boot, non garantisce il livello prima dell'esecuzione del firmware.
- Se `command.connected` e' falso oppure `shieldReady` e' falso, il loop disabilita il relay e ferma i cingoli/stepper prima di elaborare movimento, Fire o Zero.
- Fire e Zero usano lo schema "wait release": dopo fault/reconnect una pressione gia' mantenuta viene memorizzata ma non genera un fronte. Solo rilascio seguito da una nuova pressione e' operativo.
- `enterShieldFault()` e' latched: dopo NACK o lettura I2C corta il firmware inibisce relay, stepper e nuovi comandi. Non tenta un riarmo automatico della shield.

5.6 PCA, motori e inversione

- `PWMController` controlla ora `Wire.endTransmission()` e `requestFrom()`; un errore rende `isCommunicationHealthy()` falso. `setPWM()` rifiuta inoltre canali oltre 15.
- `MotorController::DCrun()` azzera PWM se il verso registrato cambia, poi scrive direzione e PWM. `DCbrake()` azzera anche il verso registrato.

- **TrackController** mantiene per ogni cingolo l'ultimo verso richiesto, il verso prima dello stop, comando pendente e istante di stop. In inversione scrive brake, attende 30 ms e applica l'ultimo comando pendente; uno stop annulla l'inversione pendente ma conserva verso e tempo per non bypassare la pausa nel percorso centro -> verso opposto.

Questo limita il colpo software di inversione ma non prova che **DCbrake()** corrisponda a frenatura fisica, non crea una rampa di velocità e non risolve il PWM DC condiviso a 50 Hz.

5.7 Controller: calibrazione, pulsanti e socket UDP

- La calibrazione usa media, minimo e massimo di 80 campioni. Ogni asse è valido solo con centro entro 512 ± 70 e spread non oltre 40; altrimenti **safeControllerData()** invia assi a 512 e pulsanti falsi fino al riavvio.
- **DRIVE_INPUT_DEADZONE=20** filtra solo il rumore del Joy1 sul controller; **TURRET_INPUT_DEADZONE=80** mantiene la sensibilità storica del Joy2. Queste soglie non sostituiscono **TRACK_COMMAND_DEADZONE=100** sul tank, applicata anche al risultato del mixing differenziale.
- **DebouncedButton** separa livello candidato e livello stabile: un cambiamento vale solo dopo 40 ms continui. Dopo boot/reconnect, **JoystickReader_requireNeutralBeforeCommands()** richiede 400 ms di quattro assi neutrali e due pulsanti rilasciati.
- **udpReady** dice che il socket locale è stato aperto. **configureUdpConnection()** controlla **udp.begin()**, richiede di nuovo neutralità e marca il primo campione della riconnessione da scartare.
- **UdpSender_send()** controlla lunghezza **snprintf**, **beginPacket**, byte di **print** e **endPacket**. Un errore rimette **udpReady** a falso e il modulo ritenta la configurazione del socket.

6 Lettura completa dei file: baseline e sorgente corrente

6.1 Configurazione PlatformIO del tank

File: tank/platformio.ini

Collegamenti audit: A-10, A-14

6.1.1 Panoramica

Questa configurazione seleziona il microcontrollore RA4M1 dell'Arduino Uno R4 WiFi, il framework Arduino e il baud rate del monitor seriale. Non dichiara dipendenze esterne: **Wire** e **WiFiS3** arrivano dal framework, mentre **motorController** e **PWMController** sono librerie locali sotto **tank/lib**. Il valore **monitor_speed=9600** deve restare coerente con **Serial.begin(9600)** nel firmware.

6.1.2 Blocchi logici

Righe	Blocco	Cosa studiare
1	Ambiente	Nomina l'ambiente di build usato dal comando PlatformIO.
2-5	Piattaforma e seriale	Sceglie package, board, framework Arduino e velocità del monitor; una versione platform non fissata può cambiare nel tempo.

6.1.3 Funzioni e failure mode

Non contiene funzioni C++. È una dichiarazione di build: PlatformIO la legge prima della compilazione per risolvere compilatore, core Arduino, bootloader e librerie.

6.1.4 Listing completo numerato

```
1 [env:uno_r4_wifi]
2 platform = renesas-ra
3 board = uno_r4_wifi
4 framework = arduino
5 monitor_speed = 9600
```

6.1.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Riga sintatticamente significativa del blocco corrente; il suo ruolo e' dettagliato dal contesto e dal listing adiacente.
2	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
3	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
4	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
5	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.

6.2 Coordinatore principale del tank

File: tank/src/main.cpp

Collegamenti audit: A-01, A-02, A-03, A-05, A-07, A-10, A-13

6.2.1 Panoramica

main.cpp non implementa tutti i movimenti direttamente: crea lo stato globale, esegue il boot e collega il comando ricevuto ai moduli di cingoli, torretta, elevazione e relay. Il modello Arduino chiama `setup()` una volta e poi richiama `loop()` senza fine. Ogni azione e' cooperativa: se una chiamata blocca il processore, anche timeout, relay e movimento attendono il ritorno nel loop.

6.2.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-24	Dipendenze e contratto	Importa API Arduino/I2C e interfacce locali; il commento documenta il routing dal pacchetto agli attuatori.
26-56	Configurazione e stato persistente	Definisce polarita' del relay, tempi, oggetto PCA9685 e timestamp/stati del relay. Gli oggetti statici esistono per tutta la vita del firmware.
58-80	Ricerca PCA9685	Sonda tre indirizzi I2C e salva il primo che risponde. Un ACK prova che qualcosa risponde, non che sia necessariamente una PCA9685.
82-112	Relay Fire	Converte lo stato logico in livello elettrico e genera un impulso sul fronte della pressione, con cooldown.
114-122	Stop centrale	Arresta cingoli tramite shield e stepper tramite D2-D5; non spegne i servo, che mantengono l'ultimo PWM.
124-149	Boot	Avvia seriale, relay, stepper, I2C, shield e AP UDP. Il ritardo iniziale e l'assenza di un enable hardware sono importanti per A-01 e A-02.
151-182	Ciclo di controllo	Legge il comando, aggiorna attuatori, applica il fail-safe software e gestisce il fronte Zero.
184-208	Telemetria	Stampa lo stato una volta al secondo; e' utile al debug ma puo' aumentare il jitter del loop.

6.2.3 Funzioni e failure mode

Funzioni. `i2cDevicePresent` invia una transazione vuota e interpreta il codice zero come ACK. `configureShieldAddress` cerca e imposta un indirizzo. `setFireRelay` applica la polarita' configurata al pin D7. `updateFireRelay` conserva il fronte precedente, misura impulso e cooldown tramite `millis`. `stopAllMotion` delega i due stop ai moduli. `setup` inizializza hardware e rete; `loop` e' lo scheduler cooperativo che collega tutte le parti.

6.2.4 Listing completo numerato

```

1 #include <Arduino.h>
2 #include <Wire.h>
3
4 #include "motorController.h"
5 #include "servoTorreta.h"
6 #include "trackController.h"
7 #include "udpReceiver.h"
8
9 /*
10  * MAIN DEL TANK
11  */

```

```

12  * Questo file non fa direttamente tutti i movimenti: coordina i moduli.
13  *
14  * Metodo di funzionamento:
15  * 1. In setup() inizializza relay, torretta, shield PCA9685 e ricevitore UDP.
16  * 2. In loop() legge l'ultimo comando arrivato dal controller ESP32.
17  * 3. Passa ogni parte del comando al modulo giusto:
18  * - driveX/driveY -> cingoli differenziali
19  * - turretX -> torretta orizzontale D2-D5
20  * - elevationY -> servo S5/S6
21  * - firePressed -> relay D7
22  * - zeroPressed -> azzeramento logico
23  * 4. Se il controller non manda pacchetti per troppo tempo, ferma i motori.
24  */
25
26  // ===== CONFIGURAZIONE RELAY SPARO =====
27
28  // Pin digitale collegato al modulo relay.
29  #define FIRE_RELAY_PIN 7
30
31  // true: relay acceso con HIGH. false: relay acceso con LOW.
32  #define FIRE_RELAY_ACTIVE_HIGH true
33
34  // Durata dell'impulso di sparo: il relay non resta acceso fisso.
35  #define FIRE_RELAY_PULSE_MS 200
36
37  // Tempo minimo tra un colpo e quello successivo.
38  #define FIRE_RELAY_COOLDOWN_MS 12000
39
40  // ===== MONITOR SERIALE =====
41
42  // Stampa le coordinate una volta al secondo per non riempire la console.
43  #define COORDINATE_PRINT_INTERVAL_MS 1000
44
45  // Shield PCA9685 condivisa da cingoli e servo di elevazione.
46  // La frequenza e' 50 Hz perche' e' compatibile con i servo.
47  static MotorController shield(50);
48
49  // Diventa true solo se la PCA9685 viene trovata su I2C.
50  static bool shieldReady = false;
51  static bool shieldFaultReported = false;
52
53  // Stato del relay sparo.
54  static bool fireRelayActive = false;
55  static unsigned long fireRelayStartTime = 0;
56  static unsigned long lastFireShotTime = 0;
57  static bool fireRelayHasShot = false;
58
59  // Verifica se un dispositivo risponde a un indirizzo I2C.
60  static bool i2cDevicePresent(uint8_t address) {
61      Wire.beginTransmission(address);
62      return Wire.endTransmission() == 0;
63  }
64
65  // Seleziona automaticamente l'indirizzo della PCA9685 della shield.
66  // Alcune shield usano indirizzi diversi, quindi il codice prova i piu' comuni.
67  static bool configureShieldAddress() {
68      static const uint8_t candidateAddresses[] = {0x60, 0x40, 0x7F};
69
70      for (uint8_t address : candidateAddresses) {
71          if (i2cDevicePresent(address)) {
72              shield.setAddress(address);
73              Serial.print("Shield PCA9685: I2C 0x");
74              Serial.println(address, HEX);
75              return true;
76          }
77      }
78
79      Serial.println("ERRORE: PCA9685 shield non rilevata");
80      return false;
81  }
82
83  // Imposta il livello elettrico corretto per il relay sparo.
84  static void setFireRelay(bool active) {
85      int level =
86          active ? (FIRE_RELAY_ACTIVE_HIGH ? HIGH : LOW) : (FIRE_RELAY_ACTIVE_HIGH ? LOW : HIGH);
87
88      digitalWrite(FIRE_RELAY_PIN, level);
89      fireRelayActive = active;
90  }

```

```

91
92 // Genera un solo impulso di sparo per ogni pressione del pulsante.
93 // Usa il fronte di salita e poi aspetta 12 secondi prima di accettare un altro colpo.
94 static void updateFireRelay(bool firePressed, bool connected) {
95     static bool lastFirePressed = false;
96     static bool fireInputArmed = false;
97
98     // Dopo timeout o perdita del collegamento il relay deve spegnersi subito.
99     // Il successivo collegamento non puo' riutilizzare una pressione rimasta attiva:
100    // prima va osservato il rilascio del pulsante Fire.
101    if (!connected) {
102        setFireRelay(false);
103        fireInputArmed = false;
104        lastFirePressed = false;
105        return;
106    }
107
108    if (!fireInputArmed) {
109        if (!firePressed) {
110            fireInputArmed = true;
111        }
112
113        // Memorizza comunque il livello ricevuto: quando Fire e' tenuto premuto
114        // alla riconnessione non verra' interpretato come fronte di salita.
115        lastFirePressed = firePressed;
116        return;
117    }
118
119    bool newFirePress = firePressed && !lastFirePressed;
120    bool cooldownFinished =
121        !fireRelayHasShot || millis() - lastFireShotTime >= FIRE_RELAY_COOLDOWN_MS;
122
123    if (newFirePress && cooldownFinished) {
124        setFireRelay(true);
125        fireRelayStartTime = millis();
126        lastFireShotTime = fireRelayStartTime;
127        fireRelayHasShot = true;
128    }
129
130    if (fireRelayActive && millis() - fireRelayStartTime >= FIRE_RELAY_PULSE_MS) {
131        setFireRelay(false);
132    }
133
134    lastFirePressed = firePressed;
135 }
136
137 // Ferma ogni movimento in caso di timeout UDP.
138 // I cingoli dipendono dalla shield; la torretta D2-D5 invece puo' essere fermata sempre.
139 static void stopAllMotion() {
140     if (shieldReady) {
141         TrackController_stop();
142     }
143
144     StepperTorretta_stop();
145 }
146
147 // Un NACK o una lettura I2C corta rende il tank disarmato fino a reset.
148 // Non tentiamo di riarmare automaticamente una PCA9685 che ha appena segnalato errore.
149 static void enterShieldFault() {
150     if (!shieldReady) {
151         return;
152     }
153
154     shieldReady = false;
155     setFireRelay(false);
156     StepperTorretta_stop();
157
158     if (!shieldFaultReported) {
159         shieldFaultReported = true;
160         Serial.println("ERRORE: comunicazione PCA9685; tank disarmato fino al reset");
161     }
162 }
163
164 void setup() {
165     // Precarica i latch GPIO nello stato sicuro prima di trasformarli in output.
166     // In questo modo D7 e D2-D5 non attendono Serial o il ritardo di avvio per
167     // ricevere il primo livello definito dal firmware.
168     int inactiveRelayLevel = FIRE_RELAY_ACTIVE_HIGH ? LOW : HIGH;
169     digitalWrite(FIRE_RELAY_PIN, inactiveRelayLevel);

```

```

170     pinMode(FIRE_RELAY_PIN, OUTPUT);
171     setFireRelay(false);
172
173     digitalWrite(TURRET_DRIVER_A_PIN, LOW);
174     digitalWrite(TURRET_DRIVER_B_PIN, LOW);
175     digitalWrite(TURRET_DRIVER_C_PIN, LOW);
176     digitalWrite(TURRET_DRIVER_D_PIN, LOW);
177     StepperTorretta_begin();
178
179     Serial.begin(9600);
180     delay(1000);
181
182     // Inizializza la PCA9685 usata da cingoli e servo elevazione.
183     Wire.begin();
184     shieldReady = configureShieldAddress();
185     if (shieldReady) {
186         // Avvia PWM, spegne le uscite motore e passa la shield ai moduli.
187         shieldReady = shield.begin();
188         if (shieldReady) {
189             shield.DCbrakeAll();
190             shieldReady = shield.isCommunicationHealthy();
191         }
192
193         if (shieldReady) {
194             TrackController_begin(shield);
195             ServoTorretta_begin(shield);
196             shieldReady = shield.isCommunicationHealthy();
197         }
198
199         if (!shieldReady) {
200             shieldFaultReported = true;
201             Serial.println("ERRORE: inizializzazione PCA9685 fallita; tank disarmato");
202         }
203     }
204
205     // Avvia WiFi access point e ricezione UDP.
206     UdpReceiver_begin();
207     Serial.println("Tank pronto");
208 }
209
210 void loop() {
211     // Riceve l'ultimo comando del controller ESP32.
212     // Se non arrivano pacchetti, UdpReceiver_update() restituisce valori sicuri centrati.
213     TankCommand command = UdpReceiver_update();
214
215     // Il rilascio di Zero deve essere osservato dopo boot o riconnessione prima
216     // di accettare un nuovo fronte di pressione.
217     static bool lastZeroPressed = false;
218     static bool zeroInputArmed = false;
219
220     // Non inviare alcun nuovo comando agli attuatori se il link e' scaduto.
221     // Spegnimento relay e arresto vengono eseguiti prima di ogni altro aggiornamento.
222     if (!command.connected || !shieldReady) {
223         setFireRelay(false);
224         stopAllMotion();
225         updateFireRelay(false, false);
226         lastZeroPressed = false;
227         zeroInputArmed = false;
228
229         if (shieldReady && !shield.isCommunicationHealthy()) {
230             enterShieldFault();
231         }
232     } else {
233         // La torretta orizzontale e' diretta: usa D2-D5, quindi non dipende dalla PCA9685.
234         StepperTorretta_updateJoystick(command.turretX);
235
236         // Cingoli ed elevazione dipendono dalla shield PWM.
237         if (shieldReady) {
238             TrackController_update(command.driveX, command.driveY);
239             if (!shield.isCommunicationHealthy()) {
240                 enterShieldFault();
241             } else {
242                 ServoTorretta_updateJoystick(command.elevationY);
243                 if (!shield.isCommunicationHealthy()) {
244                     enterShieldFault();
245                 }
246             }
247         }
248     }

```

```

249 // Fire e Zero restano inibiti se una scrittura PWM ha appena fallito.
250 if (shieldReady) {
251     updateFireRelay(command.firePressed, true);
252
253     // Zero azzerla la posizione logica della torretta una sola volta per pressione.
254     // Dopo una riconnessione, una pressione già mantenuta viene ignorata fino al rilascio.
255     if (!zeroInputArmed) {
256         if (!command.zeroPressed) {
257             zeroInputArmed = true;
258         }
259         lastZeroPressed = command.zeroPressed;
260     } else {
261         if (command.zeroPressed && !lastZeroPressed) {
262             StepperTorretta_setZero();
263             ServoTorretta_setZero();
264         }
265         lastZeroPressed = command.zeroPressed;
266     }
267 } else {
268     updateFireRelay(false, false);
269     lastZeroPressed = false;
270     zeroInputArmed = false;
271 }
272 }
273
274 // Mostra coordinate e stato connessione una volta al secondo.
275 static unsigned long lastPrintTime = 0;
276 if (millis() - lastPrintTime > COORDINATE_PRINT_INTERVAL_MS) {
277     lastPrintTime = millis();
278
279     Serial.print("RX ");
280     Serial.print(command.connected && shieldReady ? "OK" : "DISARMED");
281     Serial.print(" | drive X=");
282     Serial.print(command.driveX);
283     Serial.print(" Y=");
284     Serial.print(command.driveY);
285     Serial.print(" | turret X=");
286     Serial.print(command.turretX);
287     Serial.print(" angolo=");
288     Serial.print(StepperTorretta_getAngle());
289     Serial.print("deg | elev Y=");
290     Serial.print(command.elevationY);
291     Serial.print(" servo=");
292     Serial.print(ServoTorretta_getAngle());
293     Serial.print("deg | zero=");
294     Serial.print(command.zeroPressed ? "PREMUTO" : "off");
295     Serial.print(" fire=");
296     Serial.println(command.firePressed ? "PREMUTO" : "off");
297 }
298 }

```

6.2.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
2	Importa l'API I2C Wire usata per parlare con la PCA9685.
4	Include il modulo locale motorController.h e rende visibili la sua interfaccia e i suoi tipi.
5	Include il modulo locale servoTorretta.h e rende visibili la sua interfaccia e i suoi tipi.
6	Include il modulo locale trackController.h e rende visibili la sua interfaccia e i suoi tipi.
7	Include il modulo locale udpReceiver.h e rende visibili la sua interfaccia e i suoi tipi.
29	Configura il pin hardware tramite la macro FIRE_RELAY_PIN; il valore associato deve coincidere con il cablaggio.
32	Definisce la costante di compilazione FIRE_RELAY_ACTIVE_HIGH con valore true.

Riga	Lettura riga per riga
35	Definisce una durata nominale in millisecondi per FIRE_RELAY_PULSE_MS; viene rispettata solo quando il loop continua a girare.
38	Definisce una durata nominale in millisecondi per FIRE_RELAY_COOLDOWN_MS; viene rispettata solo quando il loop continua a girare.
43	Definisce una durata nominale in millisecondi per COORDINATE_PRINT_INTERVAL_MS; viene rispettata solo quando il loop continua a girare.
47	Dichiara l'API shield senza implementarla: la definizione e' in un file .cpp o piu' avanti.
50	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
53	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
54	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
55	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
56	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
59	Definisce la funzione i2cDevicePresent; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
60	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
61	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
66	Definisce la funzione configureShieldAddress; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
67	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
69	Avvia un ciclo con inizializzazione, condizione di prosecuzione e avanzamento espliciti.
70	Valuta una guardia condizionale prima di eseguire il blocco successivo.
71	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
72	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
73	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
74	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
78	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
79	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
83	Definisce la funzione setFireRelay; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
84	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
85	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
87	Scriva un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
88	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
93	Definisce la funzione updateFireRelay; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
94	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
96	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.

Riga	Lettura riga per riga
97	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
98	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
100	Valuta una guardia condizionale prima di eseguire il blocco successivo.
101	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
102	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
103	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
104	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
107	Valuta una guardia condizionale prima di eseguire il blocco successivo.
108	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
111	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
116	Definisce la funzione stopAllMotion; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
117	Valuta una guardia condizionale prima di eseguire il blocco successivo.
118	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
121	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
124	Definisce la funzione setup; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
125	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
126	Blocca intenzionalmente il loop per il tempo indicato; non e' un timer concorrente.
129	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
130	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
133	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
136	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
137	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
138	Valuta una guardia condizionale prima di eseguire il blocco successivo.
140	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
141	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
142	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
143	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
147	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
148	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
151	Definisce la funzione loop; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
154	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
157	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
160	Valuta una guardia condizionale prima di eseguire il blocco successivo.
161	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
162	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
165	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
169	Valuta una guardia condizionale prima di eseguire il blocco successivo.
170	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
175	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.

Riga	Lettura riga per riga
176	Valuta una guardia condizionale prima di eseguire il blocco successivo.
177	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
178	Valuta una guardia condizionale prima di eseguire il blocco successivo.
179	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
182	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
185	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
186	Valuta una guardia condizionale prima di eseguire il blocco successivo.
187	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
189	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
190	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
191	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
192	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
193	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
194	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
195	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
196	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
197	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
198	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
199	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
200	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
201	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
202	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
203	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
204	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
205	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
206	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.

6.3 Interfaccia del ricevitore UDP

File: tank/src/udpReceiver.h

Collegamenti audit: A-01, A-08, A-09, A-13, A-14

6.3.1 Panoramica

Questo header separa la rete dal resto dell'applicazione. La struct `TankCommand` e' il contratto interno: quattro assi sono in scala 0-1023, due booleani rappresentano pulsanti e `connected` segnala che un pacchetto valido e' arrivato recentemente. Non significa che il controller sia certamente presente o autenticato.

6.3.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-4, 39	Protezione header	Evita di includere la stessa dichiarazione due volte nella medesima unita' di compilazione.
14-31	Struttura TankCommand	Raccoglie il valore richiesto da ogni attuatore e lo stato del watchdog software.
33-37	API pubblica	Espone avvio dell'AP e lettura dell'ultimo comando in forma indipendente da WiFi/UDP.

6.3.3 Funzioni e failure mode

`UdpReceiver_begin` prepara AP e socket. `UdpReceiver_update` restituisce un `TankCommand` per valore: il chiamante riceve una copia coerente dell'ultimo stato memorizzato.

6.3.4 Listing completo numerato

```

1  #ifndef UDP_RECEIVER_H
2  #define UDP_RECEIVER_H
3
4  #include <Arduino.h>
5
6  /*
7   * RICEVITORE UDP DEL TANK
8   *
9   * Questo modulo crea la rete WiFi del tank e legge i comandi del controller.
10  * Il main non deve sapere come funziona WiFi/UDP: riceve solo un TankCommand
11  * gia' pronto da usare.
12  */
13
14  // Comando completo ricevuto dal controller ESP32.
15  // Tutti gli assi sono in scala 0-1023, con centro a circa 512.
16  struct TankCommand {
17      // Joy1: guida differenziale.
18      int driveX;
19      int driveY;
20
21      // Joy2: torretta orizzontale ed elevazione.
22      int turretX;
23      int elevationY;
24
25      // Pulsanti.
26      bool zeroPressed;
27      bool firePressed;
28
29      // true solo quando i pacchetti validi sono recenti e il riarmo neutro
30      // e' stato completato. false significa che nessun attuatore va comandato.
31      bool connected;
32  };
33
34  // Inizializza AP/UDP in stato disarmato; le attese tra retry non bloccano e
35  // i tentativi vengono eseguiti da UdpReceiver_update().
36  void UdpReceiver_begin();
37
38  // Legge il pacchetto piu' recente o restituisce valori sicuri se scatta il timeout.

```

```

39 TankCommand UdpReceiver_update();
40
41 #endif

```

6.3.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
16	Dichiara la struct TankCommand: un contenitore di campi che viaggiano insieme fra moduli.
18	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
19	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
22	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
23	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
26	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
27	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
30	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
34	Dichiara l'API UdpReceiver__begin senza implementarla: la definizione e' in un file .cpp o piu' avanti.
37	Dichiara l'API UdpReceiver__update senza implementarla: la definizione e' in un file .cpp o piu' avanti.
39	Chiude la include guard o il blocco di precompilazione aperto in precedenza.

6.4 AP WiFi e parser del comando UDP

File: tank/src/udpReceiver.cpp

Collegamenti audit: A-01, A-04, A-08, A-09, A-10, A-13, A-14

6.4.1 Panoramica

Il tank opera come access point e ascolta UDP sulla porta 4210. Il protocollo V1 e' testuale e compatto. Il modulo conserva `lastCommand` fra i giri di loop e lo sostituisce solo quando il parsing riesce. Dopo 500 ms senza pacchetto valido restituisce valori centrati e `connected=false`; questa e' una protezione logica, non un interruttore elettrico autonomo.

6.4.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-18	Librerie e contratto V1	Carica <code>WiFiS3</code> e <code>WiFiUdp</code> e descrive formato, scala e timeout.
20-43	Parametri e stato statico	Definisce credenziali, porta, buffer, comando sicuro iniziale e timestamp. Il buffer e' grande 64 byte ma la lettura ne usa al massimo 63.
45-61	Avvio AP	Crea l'access point, blocca per sempre se fallisce, aspetta un secondo e apre UDP.
63-99	Ricezione e parsing	Legge un datagramma, lo termina, esegue <code>sscanf</code> , limita assi e aggiorna il timestamp solo se sei campi sono letti.
101-112	Timeout e return	Ripristina valori neutrali se l'eta' dell'ultimo pacchetto supera il limite.

6.4.3 Funzioni e failure mode

`UdpReceiver_begin` effettua una sola inizializzazione dell'AP. `UdpReceiver_update` e' chiamata ad ogni giro: `parsePacket` chiede al sottosistema WiFi se esiste un datagramma, `read` copia testo nel buffer e il parser aggiorna lo stato. I fallback locali evitano di usare variabili non inizializzate, ma non rendono il parser una grammatica rigorosa.

6.4.4 Listing completo numerato

```

1  #include "udpReceiver.h"
2
3  #include <WiFiS3.h>
4  #include <WiFiUdp.h>
5
6  /*
7   * RICEVITORE UDP DEL TANK
8   *
9   * Il tank crea un access point WiFi e riceve comandi nel formato:
10  *
11  * V1;driveX;driveY;turretX;elevationY;zero;fire
12  *
13  * Per compatibilita' con la versione iniziale, il parsing usa sscanf e poi
14  * limita gli assi nella scala 0--1023.
15  */
16
17 // Rete WiFi creata dal tank.
18 const char* WIFI_SSID = "Tank_AP";
19 const char* WIFI_PASS = "12345678";
20
21 // Porta UDP condivisa da tank e controller.
22 #define UDP_PORT 4210
23
24 // Dopo questo tempo senza pacchetti validi il tank ferma i movimenti.
25 #define CONNECTION_TIMEOUT_MS 500

```

```
26
27 // Se AP o socket UDP non sono pronti, prova di nuovo con frequenza limitata.
28 #define NETWORK_RETRY_INTERVAL_MS 3000
29
30 // Prima di riarmare dopo boot, timeout o guasto rete richiede tre comandi neutrali.
31 #define REARM_NEUTRAL_PACKET_COUNT 3
32 #define REARM_NEUTRAL_TOLERANCE 20
33
34 // Valore centrale degli assi joystick in scala 0-1023.
35 #define JOYSTICK_CENTER 512
36
37 static WiFiUDP udp;
38
39 // Buffer temporaneo: viene lasciato un byte libero per il terminatore '\0'.
40 static char packetBuffer[64];
41
42 enum ReceiverNetworkState {
43     NETWORK_FAULT,
44     NETWORK_READY,
45 };
46
47 static ReceiverNetworkState networkState = NETWORK_FAULT;
48 static unsigned long lastNetworkAttemptTime = 0;
49
50 // Comando sicuro usato prima del primo pacchetto e dopo un timeout/guasto rete.
51 static TankCommand makeSafeCommand() {
52     return {JOYSTICK_CENTER, JOYSTICK_CENTER, JOYSTICK_CENTER, JOYSTICK_CENTER, false, false,
53         false};
54 }
55
56 static TankCommand lastCommand = makeSafeCommand();
57 static unsigned long lastPacketTime = 0;
58 static bool hasValidPacket = false;
59 static bool controlsArmed = false;
60 static uint8_t consecutiveNeutralPackets = 0;
61
62 // Riporta sempre il ricevitore a uno stato che non puo' comandare attuatori.
63 static void disarmControls() {
64     lastCommand = makeSafeCommand();
65     lastPacketTime = 0;
66     hasValidPacket = false;
67     controlsArmed = false;
68     consecutiveNeutralPackets = 0;
69 }
70
71 // Mettere il socket in fault non interrompe il loop: il retry verra' fatto da millis().
72 static void enterNetworkFault() {
73     networkState = NETWORK_FAULT;
74     disarmControls();
75 }
76
77 // Avvia o ripristina AP e UDP. In caso di errore ritorna false e lascia comando sicuro.
78 static bool ensureNetworkReady(unsigned long now) {
79     if (networkState == NETWORK_READY) {
80         return true;
81     }
82
83     if (now - lastNetworkAttemptTime < NETWORK_RETRY_INTERVAL_MS) {
84         return false;
85     }
86
87     Serial.println("Avvio/ripristino Access Point...");
88
89     if (WiFi.begin(WIFI_SSID, WIFI_PASS) != WL_AP_LISTENING) {
90         Serial.println("ERRORE: impossibile creare Access Point; retry programmato");
91         enterNetworkFault();
92         // Timestamp dopo la chiamata: se il coprocessore ha atteso a lungo,
93         // il giro successivo non deve ritentare immediatamente.
94         lastNetworkAttemptTime = millis();
95         return false;
96     }
97
98     if (udp.begin(UDP_PORT) == 0) {
99         Serial.println("ERRORE: impossibile aprire UDP; retry programmato");
100         enterNetworkFault();
101         lastNetworkAttemptTime = millis();
102         return false;
103     }
104 }
```

```

105     networkState = NETWORK_READY;
106     Serial.println("Access Point e UDP pronti; attesa di comandi neutrali.");
107     return true;
108 }
109
110 static bool isNeutralReleasedCommand(const TankCommand& command) {
111     return abs(command.driveX - JOYSTICK_CENTER) <= REARM_NEUTRAL_TOLERANCE &&
112            abs(command.driveY - JOYSTICK_CENTER) <= REARM_NEUTRAL_TOLERANCE &&
113            abs(command.turretX - JOYSTICK_CENTER) <= REARM_NEUTRAL_TOLERANCE &&
114            abs(command.elevationY - JOYSTICK_CENTER) <= REARM_NEUTRAL_TOLERANCE &&
115            !command.zeroPressed && !command.firePressed;
116 }
117
118 // Un pacchetto valido rinnova il watchdog, ma puo' comandare gli attuatori solo da armato.
119 static void acceptValidCommand(const TankCommand& received, unsigned long now) {
120     lastPacketTime = now;
121     hasValidPacket = true;
122
123     if (!controlsArmed) {
124         if (!isNeutralReleasedCommand(received)) {
125             consecutiveNeutralPackets = 0;
126             return;
127         }
128
129         if (consecutiveNeutralPackets < REARM_NEUTRAL_PACKET_COUNT) {
130             consecutiveNeutralPackets++;
131         }
132
133         if (consecutiveNeutralPackets < REARM_NEUTRAL_PACKET_COUNT) {
134             return;
135         }
136
137         controlsArmed = true;
138         Serial.println("Comandi riarmati dopo posizione neutra");
139     }
140
141     lastCommand = received;
142     lastCommand.connected = true;
143 }
144
145 static bool commandTimedOut(unsigned long now) {
146     return !hasValidPacket || now - lastPacketTime >= CONNECTION_TIMEOUT_MS;
147 }
148
149 // Un datagramma malformato non deve riattivare ne' alterare il comando precedente.
150 static TankCommand currentCommandOrSafe(unsigned long now) {
151     if (commandTimedOut(now)) {
152         disarmControls();
153         return makeSafeCommand();
154     }
155
156     return controlsArmed ? lastCommand : makeSafeCommand();
157 }
158
159 void UdpReceiver_begin() {
160     Serial.println("Creazione Access Point...");
161
162     disarmControls();
163     networkState = NETWORK_FAULT;
164
165     // Primo tentativo immediato dal loop; gli eventuali retry restano limitati nel tempo.
166     lastNetworkAttemptTime = millis() - NETWORK_RETRY_INTERVAL_MS;
167 }
168
169 TankCommand UdpReceiver_update() {
170     unsigned long now = millis();
171
172     // Il timeout viene valutato prima di interrogare la rete, ma non puo' sostituire un cutoff
173     // hardware se il coprocessore WiFi si blocca dentro una chiamata della libreria.
174     if (commandTimedOut(now)) {
175         disarmControls();
176     }
177
178     if (!ensureNetworkReady(now)) {
179         return makeSafeCommand();
180     }
181
182     // parsePacket() ritorna la dimensione del prossimo datagramma, oppure 0 se non ce n'e'.
183     int packetSize = udp.parsePacket();

```

```

184
185 if (packetSize > 0) {
186     // Mantiene il comportamento del parser iniziale: legge al massimo 63 byte
187     // e lascia sempre spazio per terminare la stringa con '\0'.
188     int length = udp.read(packetBuffer, sizeof(packetBuffer) - 1);
189     if (length > 0) {
190         packetBuffer[length] = '\0';
191
192         int driveX = JOYSTICK_CENTER;
193         int driveY = JOYSTICK_CENTER;
194         int turretX = JOYSTICK_CENTER;
195         int elevationY = JOYSTICK_CENTER;
196         int zero = 0;
197         int fire = 0;
198
199         // Compatibilit  V1 originale: sei interi letti da sscanf, poi clamp degli assi.
200         int parsed = sscanf(packetBuffer, "V1;%d;%d;%d;%d;%d", &driveX, &driveY, &turretX,
201                             &elevationY, &zero, &fire);
202
203         if (parsed == 6) {
204             TankCommand received = {
205                 constrain(driveX, 0, 1023),
206                 constrain(driveY, 0, 1023),
207                 constrain(turretX, 0, 1023),
208                 constrain(elevationY, 0, 1023),
209                 zero == 1,
210                 fire == 1,
211                 false,
212             };
213
214             // Il riarmo resta separato dal parser e continua a proteggere boot/reconnect.
215             acceptValidCommand(received, millis());
216         }
217     }
218 }
219
220 return currentCommandOrSafe(millis());
221 }

```

6.4.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Include il modulo locale udpReceiver.h e rende visibili la sua interfaccia e i suoi tipi.
3	Importa l'API WiFi della board corrente.
4	Importa il socket UDP fornito dal framework.
21	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
22	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
25	Fissa la porta di comunicazione UDP_PORT condivisa dai due firmware.
28	Definisce una durata nominale in millisecondi per CONNECTION_TIMEOUT_MS; viene rispettata solo quando il loop continua a girare.
31	Definisce una soglia di neutralit� del joystick per JOYSTICK_CENTER.
33	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
36	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
39	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
40	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
43	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.

Riga	Lettura riga per riga
45	Definisce la funzione UdpReceiver_begin; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
46	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
49	Valuta una guardia condizionale prima di eseguire il blocco successivo.
50	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
51	Ripete il blocco finche' la condizione resta vera; un loop infinito richiede particolare attenzione al fail-safe.
55	Blocca intenzionalmente il loop per il tempo indicato; non e' un timer concorrente.
56	Invoca il socket UDP; un successo locale non equivale a conferma di ricezione remota.
59	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
60	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
63	Definisce la funzione UdpReceiver_update; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
65	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
67	Valuta una guardia condizionale prima di eseguire il blocco successivo.
69	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
70	Valuta una guardia condizionale prima di eseguire il blocco successivo.
71	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
74	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
75	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
76	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
77	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
78	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
79	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
82	Interpreta il testo ricevuto secondo un formato; il numero di campi letti decide se il frame viene accettato.
83	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
85	Valuta una guardia condizionale prima di eseguire il blocco successivo.
87	Limita un valore all'intervallo ammesso prima dell'uso successivo.
88	Limita un valore all'intervallo ammesso prima dell'uso successivo.
89	Limita un valore all'intervallo ammesso prima dell'uso successivo.
90	Limita un valore all'intervallo ammesso prima dell'uso successivo.
91	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
92	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
93	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
96	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.

Riga	Lettura riga per riga
102	Valuta una guardia condizionale prima di eseguire il blocco successivo.
103	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
104	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
105	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
106	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
107	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
108	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
109	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
112	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.

6.5 Interfaccia e configurazione dei cingoli

File: tank/src/trackController.h

Collegamenti audit: A-11

6.5.1 Panoramica

Il modulo implementa guida differenziale per due motori DC: Joy1 Y decide avanti/indietro e Joy1 X la sterzata. Il lato sinistro e' M3, il destro M1. `TRACK_COMMAND_DEADZONE=100` e' una protezione finale del tank: viene applicata sia ai due assi prima del mixing, sia ai due comandi dei cingoli dopo `forward+/-turn`. Non e' la stessa soglia di rumore dell'ESP32. Le inversioni degli assi di guida sono definite soltanto nel controller; qui rimangono soltanto le inversioni dovute al cablaggio dei due motori.

6.5.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-24	Dipendenze e modello differenziale	Descrive assi, motori M3/M1 e formule concettuali sinistra=forward+turn, destra=forward-turn.
26-36	Scala e sicurezza comando	Fissa centro 512 e <code>TRACK_COMMAND_DEADZONE=100</code> , distinta dalla deadzone fisica del controller.
38-60	Porte, PWM e trim	Seleziona le porte motore e imposta soglie minimo/-massimo e compensazioni per i due cingoli.
62-68	Sterzo lineare e inversioni hardware	Stabilisce che Joy1 X entra direttamente nel mixing, senza modellazione aggiuntiva o soglie intermedie, e definisce i soli flag di inversione dei motori.
70-79	API	Dichiara inizializzazione, aggiornamento e stop.

6.5.3 Funzioni e failure mode

Le tre API pubbliche ricevono o conservano un riferimento al `MotorController`; l'header non compie I/O ma definisce i limiti che governano la successiva scrittura I2C. `TrackController_stop()` usa `DCbrake()` per portare il PWM a zero: il suo nome non dimostra un taglio fisico dell'alimentazione o una frenatura meccanica misurata.

6.5.4 Listing completo numerato

```

1  #ifndef TRACK_CONTROLLER_H
2  #define TRACK_CONTROLLER_H
3
4  #include <Arduino.h>
5
6  #include "motorController.h"
7
8  /*
9   * CONTROLLO CINGOLI CON MOTORI DC
10  *
11  * I motori gialli in foto sono motori DC a 2 fili, non stepper.
12  * Quindi ogni cingolo usa una sola porta motore della shield:
13  * - sinistro: M3
14  * - destro: M1
15  *
16  * M2 viene lasciato libero perche' sulla shield del progetto quella porta
17  * tende a girare anche a joystick fermo.
18  *
19  * Il joystick Joy1 viene trasformato in guida differenziale:
20  * - driveY decide avanti/indietro
21  * - driveX decide la sterzata
22  * - comando sinistro = avanti + sterzo
23  * - comando destro = avanti - sterzo
24  */
25
26  // ===== CONFIGURAZIONE GUIDA CINGOLI =====
27

```

```

28 // I joystick arrivano dal controller gia' scalati da 0 a 1023.
29 // Il centro ideale e' 512.
30 #define DRIVE_JOYSTICK_CENTER 512
31
32 // Protezione finale del comando cingolo nella scala UDP 0--1023.
33 // Viene applicata sia prima sia dopo il mixing differenziale: non deve coincidere
34 // con DRIVE_INPUT_DEADZONE dell'ESP32, che filtra soltanto il rumore del joystick.
35 // Se da fermo un lato prova ancora a muoversi, aumenta di 10/20 e riprova sul tank fisico.
36 #define TRACK_COMMAND_DEADZONE 100
37
38 // Ogni cingolo usa una porta motore DC della shield.
39 // M2 resta libero: usa M3 per il sinistro e M1 per il destro.
40 #define LEFT_TRACK_MOTOR M3
41 #define RIGHT_TRACK_MOTOR M1
42
43 // PWM minimo e massimo separato per ogni cingolo.
44 // I due motori DC non sono mai identici: uno puo' partire prima o girare piu'
45 // forte dell'altro anche con lo stesso comando.
46 //
47 // Come regolare:
48 // - un cingolo non parte ai piccoli movimenti: aumenta il suo MIN_PWM di 50;
49 // - un cingolo parte troppo presto/strappa: abbassa il suo MIN_PWM di 50;
50 // - un cingolo e' piu' veloce dell'altro: abbassa il suo MAX_PWM o PWM_PERCENT.
51 #define LEFT_TRACK_MIN_PWM 900
52 #define RIGHT_TRACK_MIN_PWM 900
53 #define LEFT_TRACK_MAX_PWM MAX_SPEED
54 #define RIGHT_TRACK_MAX_PWM MAX_SPEED
55
56 // Trim percentuale finale.
57 // 100 = normale, 90 = quel cingolo va al 90%, 110 = quel cingolo spinge di piu'.
58 // Se M1/destro corre piu' di M3/sinistro, abbassa RIGHT_TRACK_PWM_PERCENT.
59 #define LEFT_TRACK_PWM_PERCENT 100
60 #define RIGHT_TRACK_PWM_PERCENT 100
61
62 // Lo sterzo Joy1 X e' lineare: nessuna curva o soglia intermedia nascosta.
63 // La sua intensita' entra direttamente nel mixing differenziale sotto.
64
65 // L'orientamento degli assi del joystick viene corretto una sola volta sull'ESP32.
66 // Qui restano esclusivamente le inversioni dovute al cablaggio dei due motori.
67 #define LEFT_TRACK_INVERTED false
68 #define RIGHT_TRACK_INVERTED false
69
70 // Inizializza i due motori DC dei cingoli sulla shield.
71 void TrackController_begin(MotorController& controller);
72
73 // Applica il mixing differenziale lineare di Joy1 ai due cingoli.
74 void TrackController_update(int driveX, int driveY);
75
76 // Porta entrambi i cingoli a PWM zero tramite DCbrake().
77 void TrackController_stop();
78
79 #endif

```

6.5.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
6	Include il modulo locale motorController.h e rende visibili la sua interfaccia e i suoi tipi.
28-30	Documenta e definisce DRIVE_JOYSTICK_CENTER=512: e' il centro della scala UDP, non una misura diretta dell'ADC a 12 bit.
32-36	Definisce TRACK_COMMAND_DEADZONE=100. Il commento chiarisce i due punti di applicazione e perche' non va resa uguale a DRIVE_INPUT_DEADZONE dell'ESP32.
40-41	Associa M3 al cingolo sinistro e M1 al destro; un errore qui scambia il lato azionato.
51-54	Imposta PWM minimo e massimo per ciascun lato. MAX_SPEED arriva dal driver; la partenza reale va verificata con il carico meccanico.

Riga	Lettura riga per riga
59–60	Imposta il trim percentuale finale: 100 non corregge, valori diversi compensano differenze tra i motori ma possono ridurre il PWM minimo effettivo.
62–63	Documenta lo sterzo lineare: <code>driveX</code> entra direttamente nelle formule del mixing, senza percentuali o soglie intermedie da tarare.
65–68	Riserva le inversioni a M3/M1 per il cablaggio dei motori. Non esistono inversioni <code>DRIVE_X/Y</code> nel tank: Joy1 e' orientato una sola volta nell'ESP32.
71	Dichiara <code>TrackController_begin()</code> , che memorizza il riferimento al driver e parte fermo.
74	Dichiara <code>TrackController_update()</code> , chiamata dal loop del tank con <code>driveX</code> e <code>driveY</code> del frame autorizzato.
77	Dichiara <code>TrackController_stop()</code> , che richiede <code>DCbrake()</code> su entrambi i cingoli.
79	Chiude la include guard.

6.6 Mixing differenziale e PWM dei cingoli

File: tank/src/trackController.cpp

Collegamenti audit: A-11

6.6.1 Panoramica

Il file trasforma il protocollo 0–1023 in due comandi con segno. `decodeDriveAxis()` centra e filtra ciascun asse sul tank; lo sterzo entra linearmente nel mixing `forward+/-turn`; `applyTrackMotorCommand()` filtra di nuovo il comando di ogni singolo cingolo e lo traduce in direzione e PWM. Gli input sono limitati e il prodotto PWM per trim usa `long`, per cui non emerge un overflow pratico nei range correnti. L'orientamento di Joy1 non viene modificato qui: e' gia' normalizzato dall'ESP32.

6.6.2 Blocchi logici

Righe	Blocco	Cosa studiare
14–24	Driver, dead-time e vincoli	Conserva il puntatore al driver, fissa i 30 ms e verifica a compilazione che la deadzone resti interna alla scala joystick.
26–51	Stato per cingolo	Memorizza ultimo verso richiesto, verso prima dello stop, comando pendente e tempo; sceglie lo stato di M3 o M1.
53–89	Decodifica e stop	<code>decodeDriveAxis()</code> applica il filtro pre-mixing; <code>stopTrackMotor()</code> frena e conserva il contesto necessario al dead-time.
91–97	PWM lineare	Mappa direttamente la magnitudine del comando nel PWM minimo/massimo configurato.
99–182	Applicazione a un motore	Esegue filtro post-mixing, PWM, inversione di cablaggio e macchina di stato per il cambio verso.
184–213	API pubbliche	Associa il driver, calcola il mixing lineare, satura i risultati e arresta entrambi i lati.

6.6.3 Funzioni e failure mode

`stateForTrackMotor()` seleziona lo stato statico di M3 o M1. `decodeDriveAxis()` restituisce un comando firmato e applica il primo filtro `TRACK_COMMAND_DEADZONE`. `stopTrackMotor()` richiede `DCbrake()` e, se il motore era in marcia, conserva verso e `millis()` dello stop. `mapLinearTrackSpeed()` calcola il PWM in modo lineare. `applyTrackMotorCommand()` e' la macchina di stato: il filtro post-mixing e' necessario perche' `forward+/-turn` puo' lasciare piccolo un solo lato; durante i 30 ms conserva solo l'ultimo comando. Se il joystick passa dal centro al verso opposto, usa `directionBeforeStop` e `stoppedAt` per non saltare la pausa. `TrackController_update()` esegue direttamente il mixing lineare, senza modellare ulteriormente `turn`.

6.6.4 Listing completo numerato

```

1 #include "trackController.h"
2
3 /*
4  * METODO DI FUNZIONAMENTO DEI CINGOLI DC
5  *
6  * Ogni motore giallo ha solo due fili. La shield gli da:
7  * - direzione: avanti o indietro
8  * - velocita: PWM dal minimo al massimo configurato per ogni cingolo
9  *
10 * Il codice non fa piu' fasi stepper. Ora manda semplicemente PWM ai motori
11 * M1 e M3, con mixing differenziale per girare come un tank.
12 */
13
14 // Puntatore al driver PWM ricevuto dal main in TrackController_begin().

```

```

15 static MotorController* motorController = nullptr;
16
17 // Prima di invertire un cingolo, il firmware applica una breve pausa a PWM zero.
18 // Il valore va validato sul driver reale: e' un miglioramento software, non sostituisce
19 // il cutoff hardware in caso di blocco del bus I2C.
20 #define TRACK_REVERSE_DEAD_TIME_MS 30
21
22 // La deadzone deve lasciare almeno un comando positivo e uno negativo disponibili.
23 static_assert(TRACK_COMMAND_DEADZONE > 0 && TRACK_COMMAND_DEADZONE < DRIVE_JOYSTICK_CENTER,
24             "TRACK_COMMAND_DEADZONE must stay inside the joystick range");
25
26 struct TrackMotorState {
27     // Ultimo verso passato a DCRun(). E' una richiesta software, non una misura del motore.
28     int lastRequestedDirection;
29     // Ultimo verso prima di DCbrake(). Resta memorizzato anche se il joystick torna al centro,
30     // cosi' un'inversione immediata rispetta comunque il tempo di pausa.
31     int directionBeforeStop;
32     int pendingDirection;
33     int pendingSpeed;
34     bool waitingForReverse;
35     unsigned long stoppedAt;
36 };
37
38 static TrackMotorState leftTrackState = {0, 0, 0, 0, false, 0};
39 static TrackMotorState rightTrackState = {0, 0, 0, 0, false, 0};
40
41 static TrackMotorState* stateForTrackMotor(int motorNumber) {
42     if (motorNumber == LEFT_TRACK_MOTOR) {
43         return &leftTrackState;
44     }
45
46     if (motorNumber == RIGHT_TRACK_MOTOR) {
47         return &rightTrackState;
48     }
49
50     return nullptr;
51 }
52
53 // Decodifica un asse UDP 0--1023 in un comando firmato centrato su zero.
54 // Esempio: 512 diventa 0, 1023 diventa circa +511, 0 diventa -512.
55 // L'inversione dell'asse e' responsabile' del controller ESP32, non del tank.
56 static int decodeDriveAxis(int value) {
57     int centered = constrain(value, 0, 1023) - DRIVE_JOYSTICK_CENTER;
58
59     // Primo filtro lato tank: protegge anche da sender UDP diversi dall'ESP32.
60     // Il filtro finale di applyTrackMotorCommand() resta necessario dopo forward +/- turn.
61     if (abs(centered) < TRACK_COMMAND_DEADZONE) {
62         centered = 0;
63     }
64
65     return centered;
66 }
67
68 // Porta un cingolo a PWM zero e annulla un'eventuale inversione in attesa.
69 static void stopTrackMotor(int motorNumber) {
70     if (motorController == nullptr) {
71         return;
72     }
73
74     TrackMotorState* state = stateForTrackMotor(motorNumber);
75     motorController->DCbrake(motorNumber);
76
77     if (state != nullptr) {
78         // Non azzerare directionBeforeStop: serve se dopo il centro viene chiesto il verso opposto.
79         if (state->lastRequestedDirection != 0) {
80             state->directionBeforeStop = state->lastRequestedDirection;
81             state->stoppedAt = millis();
82         }
83
84         state->lastRequestedDirection = 0;
85         state->pendingDirection = 0;
86         state->pendingSpeed = 0;
87         state->waitingForReverse = false;
88     }
89 }
90
91 // Mappa avanti/indietro in modo lineare: Joy1 Y deve poter arrivare al massimo.
92 static int mapLinearTrackSpeed(int magnitude, int minPwm, int maxPwm) {
93     int fullMagnitude = DRIVE_JOYSTICK_CENTER - 1;

```

```

94     int limitedMagnitude = constrain(magnitude, TRACK_COMMAND_DEADZONE, fullMagnitude);
95
96     return map(limitedMagnitude, TRACK_COMMAND_DEADZONE, fullMagnitude, minPwm, maxPwm);
97 }
98
99 // Applica un comando firmato a un cingolo, includendo PWM e pausa sicura di inversione.
100 static void applyTrackMotorCommand(int motorNumber, bool inverted, int command, int minPwm,
101                                   int maxPwm, int pwmPercent) {
102     if (motorController == nullptr) {
103         return;
104     }
105
106     TrackMotorState* state = stateForTrackMotor(motorNumber);
107     if (state == nullptr) {
108         return;
109     }
110
111     command = constrain(command, -DRIVE_JOYSTICK_CENTER, DRIVE_JOYSTICK_CENTER);
112     minPwm = constrain(minPwm, 0, MAX_SPEED);
113     maxPwm = constrain(maxPwm, minPwm, MAX_SPEED);
114     pwmPercent = constrain(pwmPercent, 0, 150);
115
116     // Secondo filtro lato tank: il mixing puo' lasciare piccolo un solo cingolo anche
117     // quando forward e turn erano entrambi fuori dalla deadzone individuale.
118     if (abs(command) < TRACK_COMMAND_DEADZONE) {
119         stopTrackMotor(motorNumber);
120         return;
121     }
122
123     int magnitude = abs(command);
124
125     // Avanti/indietro e il comando finale dei motori restano lineari.
126     int speed = mapLinearTrackSpeed(magnitude, minPwm, maxPwm);
127     speed = constrain(static_cast<int>((static_cast<long>(speed) * pwmPercent) / 100), 0, MAX_SPEED);
128
129     int direction = command > 0 ? FORWARD : BACKWARD;
130     if (inverted) {
131         direction = direction == FORWARD ? BACKWARD : FORWARD;
132     }
133
134     unsigned long now = millis();
135
136     // Durante la pausa di inversione conserva soltanto l'ultimo comando,
137     // senza riaccendere il ponte H prima che il tempo minimo sia trascorso.
138     if (state->waitingForReverse) {
139         state->pendingDirection = direction;
140         state->pendingSpeed = speed;
141
142         if (now - state->stoppedAt < TRACK_REVERSE_DEAD_TIME_MS) {
143             return;
144         }
145
146         motorController->DCrun(motorNumber, state->pendingDirection, state->pendingSpeed);
147         state->lastRequestedDirection = state->pendingDirection;
148         state->directionBeforeStop = state->pendingDirection;
149         state->waitingForReverse = false;
150         return;
151     }
152
153     // Non invertire direttamente un motore in movimento: frena, attendi e poi riapplica.
154     if (state->lastRequestedDirection != 0 && state->lastRequestedDirection != direction) {
155         motorController->DCbrake(motorNumber);
156         state->directionBeforeStop = state->lastRequestedDirection;
157         state->lastRequestedDirection = 0;
158         state->pendingDirection = direction;
159         state->pendingSpeed = speed;
160         state->waitingForReverse = true;
161         state->stoppedAt = now;
162         return;
163     }
164
165     // Se e' gia' stato fermato, anche il percorso centro -> verso opposto deve attendere.
166     // Senza questo controllo il vecchio codice poteva bypassare i 30 ms dopo il centro.
167     if (state->lastRequestedDirection == 0 && state->directionBeforeStop != 0 &&
168         state->directionBeforeStop != direction &&
169         now - state->stoppedAt < TRACK_REVERSE_DEAD_TIME_MS) {
170         state->pendingDirection = direction;
171         state->pendingSpeed = speed;
172         state->waitingForReverse = true;

```



```

173     return;
174 }
175
176 // Il comando viene riscritto anche se invariato: non fare caching qui senza
177 // mantenere una transazione I2C periodica, altrimenti un guasto del bus puo'
178 // restare invisibile al controllo di salute della shield.
179 motorController->DCrun(motorNumber, direction, speed);
180 state->lastRequestedDirection = direction;
181 state->directionBeforeStop = direction;
182 }
183
184 void TrackController_begin(MotorController& controller) {
185     // Salva il riferimento alla shield inizializzata dal main.
186     motorController = &controller;
187
188     // Parte sempre con entrambi i cingoli spenti.
189     TrackController_stop();
190 }
191
192 void TrackController_update(int driveX, int driveY) {
193     // Joy1 Y: avanti/indietro. Joy1 X: sterzata.
194     int forward = decodeDriveAxis(driveY);
195     int turn = decodeDriveAxis(driveX);
196
197     // Mixing differenziale:
198     // - avanti dritto: entrambi i motori girano uguali
199     // - sterzata: un lato accelera e l'altro rallenta
200     // - solo X: un motore avanti e uno indietro, quindi ruota sul posto
201     int leftCommand = constrain(forward + turn, -DRIVE_JOYSTICK_CENTER, DRIVE_JOYSTICK_CENTER);
202     int rightCommand = constrain(forward - turn, -DRIVE_JOYSTICK_CENTER, DRIVE_JOYSTICK_CENTER);
203
204     applyTrackMotorCommand(LEFT_TRACK_MOTOR, LEFT_TRACK_INVERTED, leftCommand, LEFT_TRACK_MIN_PWM,
205                             LEFT_TRACK_MAX_PWM, LEFT_TRACK_PWM_PERCENT);
206     applyTrackMotorCommand(RIGHT_TRACK_MOTOR, RIGHT_TRACK_INVERTED, rightCommand,
207                             RIGHT_TRACK_MIN_PWM, RIGHT_TRACK_MAX_PWM, RIGHT_TRACK_PWM_PERCENT);
208 }
209
210 void TrackController_stop() {
211     stopTrackMotor(LEFT_TRACK_MOTOR);
212     stopTrackMotor(RIGHT_TRACK_MOTOR);
213 }

```

6.6.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Include il modulo locale trackController.h e rende visibili la sua interfaccia e i suoi tipi.
14-15	Dichiara il puntatore statico motorController. Resta valido per l'intera esecuzione ma vale nullptr finche' TrackController_begin() non riceve il driver dal main.
17-20	Definisce TRACK_REVERSE_DEAD_TIME_MS=30. E' un intervallo software misurato con millis(), non una garanzia che il ponte H sia fisicamente diseccitato.
22-24	static_assert ferma la compilazione se TRACK_COMMAND_DEADZONE non resta strettamente dentro la semiscala joystick, lasciando almeno un comando positivo e uno negativo.
27-37	Dichiara TrackMotorState. lastRequestedDirection e' l'ultimo verso passato a DCrun(), non un feedback fisico; directionBeforeStop e stoppedAt sopravvivono allo stop; i campi pending* memorizzano l'ultimo comando durante la pausa.
39-40	Inizializza stati indipendenti per M3 sinistro e M1 destro. Nessuna allocazione dinamica avviene nel loop.
42-52	stateForTrackMotor() converte il numero della porta nella struct corretta e restituisce nullptr per una porta non prevista, evitando di dereferenziare stato inesistente.
54-58	decodeDriveAxis() limita l'input UDP a 0-1023 e sottrae 512, ottenendo circa -512..+511. Non inverte Joy1: l'inversione dell'asse appartiene all'ESP32.

Riga	Lettura riga per riga
60–66	Applica il primo filtro <code>TRACK_COMMAND_DEADZONE=100</code> ; un sender UDP alternativo non puo' quindi generare piccoli comandi continui prima del mixing.
69–76	<code>stopTrackMotor()</code> controlla il driver, trova lo stato e invia <code>DCbrake()</code> al motore. La chiamata porta il PWM a zero secondo l'implementazione del driver.
79–89	Se era attivo un verso, salva verso e istante dello stop; azzera solo il verso correntemente richiesto e l'eventuale comando pendente. Questa conservazione rende sicuro il percorso centro -> verso opposto.
93–98	<code>mapLinearTrackSpeed()</code> limita la magnitudine alla porzione utile e la mappa tra PWM minimo e massimo. Un PWM minimo sufficiente a far partire il motore resta una misura hardware.
99–114	<code>applyTrackMotorCommand()</code> riceve porta, inversione di cablaggio, comando firmato e limiti PWM; controlla puntatore/stato e limita tutti gli input prima dell'I/O.
116–121	Applica il secondo filtro da 100 dopo il mixing. E' distinto ma necessario: <code>forward+turn</code> o <code>forward-turn</code> possono produrre un residuo piccolo su un solo cingolo.
123–132	Converte magnitudine in PWM lineare, usa <code>long</code> nel prodotto con il trim, ricava <code>FORWARD/BACKWARD</code> e applica solo l'inversione elettrica del singolo motore.
134–150	Durante un'inversione gia' in attesa aggiorna solo l'ultimo comando pendente. Finche' <code>now-stoppedAt<30</code> , non richiama <code>DCrun()</code> ; allo scadere applica il comando piu' recente. La sottrazione unsigned resta valida al rollover di <code>millis()</code> .
153–163	Se il motore e' in marcia e il verso cambia, invia <code>DCbrake()</code> , salva il verso precedente e avvia la pausa. Non inverte direttamente il ponte H.
165–174	Copre il caso prima assente: dopo joystick al centro, una richiesta immediata del verso opposto confronta <code>directionBeforeStop</code> e aspetta i 30 ms rimanenti invece di ripartire subito.
176–182	Riscrive <code>DCrun()</code> anche a comando invariato. Il commento vieta un caching ingenuo: una transazione I2C periodica aiuta il controllo di salute a osservare un guasto del bus.
184–190	<code>TrackController_begin()</code> memorizza il riferimento al driver gia' inizializzato dal main e avvia il modulo con i cingoli fermati.
192–208	<code>TrackController_update()</code> decodifica Y e X, calcola direttamente <code>left=forward+turn</code> e <code>right=forward-turn</code> , limita i risultati e li invia alle due macchine di stato.
210–212	<code>TrackController_stop()</code> richiede lo stop di entrambi i lati; non disalimenta materialmente la shield.

6.7 Interfaccia torretta ed elevazione

File: tank/src/servoTorretta.h

Collegamenti audit: A-07, A-11

6.7.1 Panoramica

Il file riunisce due attuatori diversi: la rotazione orizzontale usa quattro GPIO diretti verso un driver stepper, mentre l'elevazione usa due servo tramite PCA9685. Le API distinguono **StepperTorretta** e **ServoTorretta**; il nome comune non implica lo stesso hardware.

6.7.2 Blocchi logici

Righe	Blocco	Cosa studiare
1–21	Due sottosistemi	Introduce stepper D2–D5 e servo S5/S6 come percorsi separati.
23–38	Costanti stepper	Definisce passi per giro, pin, intervallo e deadzone.
40–68	API stepper e servo	Dichiara operazioni di avvio, movimento, zero, stop e lettura dello stato.

6.7.3 Funzioni e failure mode

Per lo stepper, Zero resetta soltanto il conteggio software. Per i servo, **ServoTorretta_setZero** delega a una posizione fisica minima. Questa differenza e' fondamentale per capire A-07.

6.7.4 Listing completo numerato

```

1  #ifndef SERVO_TORRETA_H
2  #define SERVO_TORRETA_H
3
4  #include <Arduino.h>
5
6  #include "motorController.h"
7
8  /*
9   * TORRETTA
10  *
11  * Questo modulo gestisce due movimenti diversi:
12  *
13  * 1. Torretta orizzontale:
14  * - e' uno stepper con driver esterno
15  * - i segnali del driver sono collegati ai pin digitali D2-D5
16  * - usa la sequenza A, B, C, D come nel vecchio codice che funzionava
17  *
18  * 2. Elevazione:
19  * - usa due servo collegati alla shield su S5 e S6
20  * - i due servo si muovono specchiati per alzare/abbassare insieme
21  */
22
23 // ===== CONFIGURAZIONE TORRETTA ORIZZONTALE =====
24
25 // Numero di step usato per convertire la posizione logica della torretta in gradi.
26 #define STEPS_PER_REV 2048
27
28 // Mappa fisica: driver torretta A/B/C/D collegato alle porte digitali della shield.
29 #define TURRET_DRIVER_A_PIN 2
30 #define TURRET_DRIVER_B_PIN 3
31 #define TURRET_DRIVER_C_PIN 4
32 #define TURRET_DRIVER_D_PIN 5
33
34 // Piu' alto significa torretta piu' lenta.
35 #define TURRET_STEP_INTERVAL_MS 2
36
37 // Valori joystick vicino al centro che non muovono torretta o elevazione.
38 #define TURRET_JOYSTICK_DEADZONE 200
39

```

```

40 // Inizializza lo stepper orizzontale della torretta.
41 void StepperTorretta_begin();
42
43 // Legge Joy2 X: sotto centro gira a sinistra, sopra centro gira a destra.
44 void StepperTorretta_updateJoystick(int joystickValue);
45
46 // Azzerà solo il conteggio logico, non muove fisicamente il motore.
47 void StepperTorretta_setZero();
48
49 // Spegne tutti i segnali A/B/C/D del driver.
50 void StepperTorretta_stop();
51
52 // Ritorna l'angolo logico calcolato dagli step fatti.
53 int StepperTorretta_getAngle();
54
55 // Inizializza i due servo che alzano e abbassano la torretta.
56 void ServoTorretta_begin(MotorController& controller);
57
58 // Legge Joy2 Y e cambia gradualmente l'angolo dei servo.
59 void ServoTorretta_updateJoystick(int joystickValue);
60
61 // Imposta direttamente l'angolo di elevazione, rispettando i limiti.
62 void ServoTorretta_setAngle(int angle);
63
64 // Porta l'elevazione alla posizione logica minima.
65 void ServoTorretta_setZero();
66
67 // Ritorna l'angolo logico attuale dei servo.
68 int ServoTorretta_getAngle();
69
70 #endif

```

6.7.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
6	Include il modulo locale motorController.h e rende visibili la sua interfaccia e i suoi tipi.
26	Definisce un limite o una conversione di posizione per STEPS_PER_REV.
29	Configura il pin hardware tramite la macro TURRET_DRIVER_A_PIN; il valore associato deve coincidere con il cablaggio.
30	Configura il pin hardware tramite la macro TURRET_DRIVER_B_PIN; il valore associato deve coincidere con il cablaggio.
31	Configura il pin hardware tramite la macro TURRET_DRIVER_C_PIN; il valore associato deve coincidere con il cablaggio.
32	Configura il pin hardware tramite la macro TURRET_DRIVER_D_PIN; il valore associato deve coincidere con il cablaggio.
35	Definisce una durata nominale in millisecondi per TURRET_STEP_INTERVAL_MS; viene rispettata solo quando il loop continua a girare.
38	Definisce una soglia di neutralita' del joystick per TURRET_JOYSTICK_DEADZONE.
41	Dichiara l'API StepperTorretta_begin senza implementarla: la definizione e' in un file .cpp o piu' avanti.
44	Dichiara l'API StepperTorretta_updateJoystick senza implementarla: la definizione e' in un file .cpp o piu' avanti.
47	Dichiara l'API StepperTorretta_setZero senza implementarla: la definizione e' in un file .cpp o piu' avanti.
50	Dichiara l'API StepperTorretta_stop senza implementarla: la definizione e' in un file .cpp o piu' avanti.

Riga	Lettura riga per riga
53	Dichiara l'API StepperTorretta_getAngle senza implementarla: la definizione e' in un file .cpp o piu' avanti.
56	Dichiara l'API ServoTorretta_begin senza implementarla: la definizione e' in un file .cpp o piu' avanti.
59	Dichiara l'API ServoTorretta_updateJoystick senza implementarla: la definizione e' in un file .cpp o piu' avanti.
62	Dichiara l'API ServoTorretta_setAngle senza implementarla: la definizione e' in un file .cpp o piu' avanti.
65	Dichiara l'API ServoTorretta_setZero senza implementarla: la definizione e' in un file .cpp o piu' avanti.
68	Dichiara l'API ServoTorretta_getAngle senza implementarla: la definizione e' in un file .cpp o piu' avanti.
70	Chiude la include guard o il blocco di precompilazione aperto in precedenza.

6.8 Sequenza stepper e controllo dei servo

File: tank/src/servoTorretta.cpp

Collegamenti audit: A-06, A-07, A-11

6.8.1 Panoramica

La rotazione conserva fase e conteggio software, e alimenta una sola bobina alla volta in sequenza A-B-C-D. L'elevazione conserva un angolo e lo aggiorna di un grado ogni 30 ms. Nessuno dei due percorsi dispone di feedback: i limiti e le coordinate sono creduti dal software, non misurati dalla meccanica.

6.8.2 Blocchi logici

Righe	Blocco	Cosa studiare
18-53	Limiti e stato	Definisce angoli, canali, intervalli, conversione step/gradi e variabili persistenti.
55-86	Scrittura di fase	Attiva esattamente una combinazione A/B/C/D e segna il driver come abilitato.
88-110	Step destro/sinistro	Controlla limite logico, aggiorna indice modulo quattro e cambia conteggio.
112-167	Lifecycle stepper	Configura pin, limita ritmo con <code>millis</code> , resetta soltanto lo stato logico, spegne bobine e calcola angolo.
169-216	Lifecycle servo	Memorizza shield, aggiorna lentamente angolo e invia coppia specchiata ai due canali.

6.8.3 Funzioni e failure mode

`writeTurretStep` porta fisicamente i quattro pin al pattern scelto. `stepRight` e `stepLeft` cambiano fase e posizione, ma non possono sapere se il motore ha realmente girato. `StepperTorretta_updateJoystick` fa al massimo uno step per giro del loop se sono trascorsi 2 ms. `ServoTorretta_setAngle` limita 0-47 e comanda la coppia. A 2 ms il massimo teorico e' 500 step/s, circa 14,6 rpm assumendo 2048 step/giro; WiFi/I2C/seriale possono solo ridurlo.

6.8.4 Listing completo numerato

```

1  #include "servoTorretta.h"
2
3  /*
4   * METODO DI FUNZIONAMENTO DELLA TORRETTA
5   *
6   * Torretta orizzontale:
7   * - il driver esterno riceve quattro segnali: A, B, C, D
8   * - ad ogni step accendiamo una sola uscita alla volta
9   * - la sequenza in avanti e': A -> B -> C -> D -> A
10  * - la sequenza indietro e': D -> C -> B -> A -> D
11  *
12  * Elevazione:
13  * - i due servo sono collegati alla PCA9685 della shield su S5 e S6
14  * - il primo servo riceve l'angolo normale
15  * - il secondo servo riceve un angolo specchiato: ELEVATION_MIRROR_BASE - angolo
16  */
17
18  // ===== LIMITI MECCANICI =====
19
20  // La torretta orizzontale si ferma a mezzo giro per lato.
21  #define TURRET_MIN_ANGLE -180
22  #define TURRET_MAX_ANGLE 180
23
24  // I due servo di elevazione non devono superare questo intervallo meccanico.
25  // Range richiesto per la torretta: 0 gradi = basso, 47 gradi = alto.
26  #define ELEVATION_MIN_ANGLE 0
27  #define ELEVATION_MAX_ANGLE 47

```

```
28 #define ELEVATION_MIRROR_BASE 90
29 #define ELEVATION_SERVO_A S5
30 #define ELEVATION_SERVO_B S6
31
32 // Centro logico dei joystick dopo conversione 0-1023.
33 #define JOYSTICK_CENTER 512
34
35 // I servo vengono aggiornati piu' lentamente dello stepper per restare stabili.
36 #define SERVO_INTERVAL_MS 30
37
38 // Conversione dei limiti in gradi nei limiti in step.
39 #define TURRET_MIN_STEPS ((STEPS_PER_REV * TURRET_MIN_ANGLE) / 360)
40 #define TURRET_MAX_STEPS ((STEPS_PER_REV * TURRET_MAX_ANGLE) / 360)
41
42 // Puntatore alla shield ricevuta dal main. Serve solo per i servo S5/S6.
43 static MotorController* shield = nullptr;
44
45 // Stato logico dello stepper orizzontale.
46 static int currentSteps = 0;
47 static int stepIndex = 0;
48 static unsigned long lastStepTime = 0;
49 static bool stepperEnabled = false;
50
51 // Stato logico dei servo di elevazione.
52 static int currentServoAngle = 0;
53 static unsigned long lastServoUpdateTime = 0;
54
55 // Scrive fisicamente una fase sul driver della torretta.
56 // Questa e' la stessa sequenza a una bobina del vecchio codice diretto Arduino.
57 static void writeTurretStep(int phase) {
58     switch (phase) {
59         case 0:
60             digitalWrite(TURRET_DRIVER_A_PIN, HIGH);
61             digitalWrite(TURRET_DRIVER_B_PIN, LOW);
62             digitalWrite(TURRET_DRIVER_C_PIN, LOW);
63             digitalWrite(TURRET_DRIVER_D_PIN, LOW);
64             break;
65         case 1:
66             digitalWrite(TURRET_DRIVER_A_PIN, LOW);
67             digitalWrite(TURRET_DRIVER_B_PIN, HIGH);
68             digitalWrite(TURRET_DRIVER_C_PIN, LOW);
69             digitalWrite(TURRET_DRIVER_D_PIN, LOW);
70             break;
71         case 2:
72             digitalWrite(TURRET_DRIVER_A_PIN, LOW);
73             digitalWrite(TURRET_DRIVER_B_PIN, LOW);
74             digitalWrite(TURRET_DRIVER_C_PIN, HIGH);
75             digitalWrite(TURRET_DRIVER_D_PIN, LOW);
76             break;
77         case 3:
78             digitalWrite(TURRET_DRIVER_A_PIN, LOW);
79             digitalWrite(TURRET_DRIVER_B_PIN, LOW);
80             digitalWrite(TURRET_DRIVER_C_PIN, LOW);
81             digitalWrite(TURRET_DRIVER_D_PIN, HIGH);
82             break;
83     }
84
85     stepperEnabled = true;
86 }
87
88 // Fa uno step verso destra, se non siamo al limite software.
89 static void stepRight() {
90     if (currentSteps >= TURRET_MAX_STEPS) {
91         StepperTorretta_stop();
92         return;
93     }
94
95     stepIndex = (stepIndex + 1) % 4;
96     writeTurretStep(stepIndex);
97     currentSteps++;
98 }
99
100 // Fa uno step verso sinistra, se non siamo al limite software.
101 static void stepLeft() {
102     if (currentSteps <= TURRET_MIN_STEPS) {
103         StepperTorretta_stop();
104         return;
105     }
106 }
```

```

107     stepIndex = (stepIndex + 3) % 4;
108     writeTurretStep(stepIndex);
109     currentSteps--;
110 }
111
112 void StepperTorretta_begin() {
113     // I pin D2-D5 escono dalla shield come pin digitali Arduino.
114     pinMode(TURRET_DRIVER_A_PIN, OUTPUT);
115     pinMode(TURRET_DRIVER_B_PIN, OUTPUT);
116     pinMode(TURRET_DRIVER_C_PIN, OUTPUT);
117     pinMode(TURRET_DRIVER_D_PIN, OUTPUT);
118
119     // Forza subito tutte le uscite a LOW e riparte dalla fase 0.
120     stepperEnabled = true;
121     StepperTorretta_stop();
122     currentSteps = 0;
123     stepIndex = 0;
124 }
125
126 void StepperTorretta_updateJoystick(int joystickValue) {
127     unsigned long now = millis();
128
129     // Limita la velocita: se non e' passato abbastanza tempo, non fa step.
130     if (now - lastStepTime < TURRET_STEP_INTERVAL_MS) {
131         return;
132     }
133
134     lastStepTime = now;
135
136     // Joy2 X sotto il centro gira a sinistra, sopra il centro gira a destra.
137     // Dentro la deadzone spegne il driver per evitare vibrazione.
138     if (joystickValue < JOYSTICK_CENTER - TURRET_JOYSTICK_DEADZONE) {
139         stepLeft();
140     } else if (joystickValue > JOYSTICK_CENTER + TURRET_JOYSTICK_DEADZONE) {
141         stepRight();
142     } else {
143         StepperTorretta_stop();
144     }
145 }
146
147 void StepperTorretta_setZero() {
148     // Azzerla la posizione logica senza muovere il motore.
149     currentSteps = 0;
150 }
151
152 void StepperTorretta_stop() {
153     if (!stepperEnabled) {
154         return;
155     }
156
157     // Tutte le uscite basse: il driver non alimenta nessuna bobina.
158     digitalWrite(TURRET_DRIVER_A_PIN, LOW);
159     digitalWrite(TURRET_DRIVER_B_PIN, LOW);
160     digitalWrite(TURRET_DRIVER_C_PIN, LOW);
161     digitalWrite(TURRET_DRIVER_D_PIN, LOW);
162     stepperEnabled = false;
163 }
164
165 int StepperTorretta_getAngle() {
166     return (currentSteps * 360L) / STEPS_PER_REV;
167 }
168
169 void ServoTorretta_begin(MotorController& controller) {
170     // Salva la shield e porta i servo alla posizione iniziale.
171     shield = &controller;
172     ServoTorretta_setZero();
173 }
174
175 void ServoTorretta_updateJoystick(int joystickValue) {
176     unsigned long now = millis();
177
178     // I servo non vengono aggiornati ad ogni loop: 30 ms e' piu' stabile.
179     if (now - lastServoUpdateTime < SERVO_INTERVAL_MS) {
180         return;
181     }
182
183     lastServoUpdateTime = now;
184
185     int nextAngle = currentServoAngle;

```



```

186 // Joy2 Y abbassa/alza di un grado alla volta.
187 if (joystickValue < JOYSTICK_CENTER - TURRET_JOYSTICK_DEADZONE) {
188     nextAngle--;
189 } else if (joystickValue > JOYSTICK_CENTER + TURRET_JOYSTICK_DEADZONE) {
190     nextAngle++;
191 }
192
193 if (nextAngle != currentServoAngle) {
194     ServoTorretta_setAngle(nextAngle);
195 }
196 }
197
198 void ServoTorretta_setAngle(int angle) {
199     if (shield == nullptr) {
200         return;
201     }
202
203     // Blocca l'angolo dentro i limiti meccanici e comanda i due servo specchiati.
204     currentServoAngle = constrain(angle, ELEVATION_MIN_ANGLE, ELEVATION_MAX_ANGLE);
205     shield->servoPairTurn(ELEVATION_SERVO_A, ELEVATION_SERVO_B, currentServoAngle,
206                          ELEVATION_MIRROR_BASE);
207 }
208
209 void ServoTorretta_setZero() {
210     ServoTorretta_setAngle(ELEVATION_MIN_ANGLE);
211 }
212
213 int ServoTorretta_getAngle() {
214     return currentServoAngle;
215 }
216

```

6.8.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Include il modulo locale servoTorretta.h e rende visibili la sua interfaccia e i suoi tipi.
21	Definisce un limite o una conversione di posizione per TURRET_MIN_ANGLE.
22	Definisce un limite o una conversione di posizione per TURRET_MAX_ANGLE.
26	Definisce un limite o una conversione di posizione per ELEVATION_MIN_ANGLE.
27	Definisce un limite o una conversione di posizione per ELEVATION_MAX_ANGLE.
28	Definisce la costante di compilazione ELEVATION_MIRROR_BASE con valore 90.
29	Definisce la costante di compilazione ELEVATION_SERVO_A con valore S5.
30	Definisce la costante di compilazione ELEVATION_SERVO_B con valore S6.
33	Definisce una soglia di neutralità del joystick per JOYSTICK_CENTER.
36	Definisce una durata nominale in millisecondi per SERVO_INTERVAL_MS; viene rispettata solo quando il loop continua a girare.
39	Definisce un limite o una conversione di posizione per TURRET_MIN_STEPS.
40	Definisce un limite o una conversione di posizione per TURRET_MAX_STEPS.
43	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
46	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
47	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
48	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
49	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.

Riga	Lettura riga per riga
52	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
53	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
57	Definisce la funzione writeTurretStep; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
58	Apri una selezione multi-ramo basata sul valore dell'espressione.
59	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
60	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
61	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
62	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
63	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
64	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
65	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
66	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
67	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
68	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
69	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
70	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
71	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
72	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
73	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
74	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
75	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
76	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
77	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
78	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
79	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
80	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
81	Scrivi un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
82	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
85	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.

Riga	Lettura riga per riga
89	Definisce la funzione stepRight; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
90	Valuta una guardia condizionale prima di eseguire il blocco successivo.
91	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
92	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
95	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
96	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
97	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
101	Definisce la funzione stepLeft; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
102	Valuta una guardia condizionale prima di eseguire il blocco successivo.
103	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
104	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
107	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
108	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
109	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
112	Definisce la funzione StepperTorretta_begin; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
114	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
115	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
116	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
117	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
120	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
121	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
122	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
123	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
126	Definisce la funzione StepperTorretta_updateJoystick; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
127	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
130	Valuta una guardia condizionale prima di eseguire il blocco successivo.
131	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
134	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
138	Valuta una guardia condizionale prima di eseguire il blocco successivo.
139	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
140	Prosegue la catena condizionale: questo ramo e' valutato solo se quelli precedenti non sono stati scelti.
141	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
142	Apri l'alternativa della condizione precedente.
143	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
147	Definisce la funzione StepperTorretta_setZero; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
149	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
152	Definisce la funzione StepperTorretta_stop; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
153	Valuta una guardia condizionale prima di eseguire il blocco successivo.
154	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.

Riga	Lettura riga per riga
158	Scrivo un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
159	Scrivo un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
160	Scrivo un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
161	Scrivo un livello logico sul GPIO e quindi ha un effetto elettrico diretto sul driver/relay.
162	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
165	Definisce la funzione StepperTorretta_getAngle; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
166	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
169	Definisce la funzione ServoTorretta_begin; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
171	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
172	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
175	Definisce la funzione ServoTorretta_updateJoystick; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
176	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
179	Valuta una guardia condizionale prima di eseguire il blocco successivo.
180	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
183	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
185	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
188	Valuta una guardia condizionale prima di eseguire il blocco successivo.
189	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
190	Prosegue la catena condizionale: questo ramo e' valutato solo se quelli precedenti non sono stati scelti.
191	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
194	Valuta una guardia condizionale prima di eseguire il blocco successivo.
195	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
199	Definisce la funzione ServoTorretta_setAngle; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
200	Valuta una guardia condizionale prima di eseguire il blocco successivo.
201	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
205	Limita un valore all'intervallo ammesso prima dell'uso successivo.
206	Delega il comando al controller shield tramite il puntatore precedentemente inizializzato.
207	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
210	Definisce la funzione ServoTorretta_setZero; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
211	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
214	Definisce la funzione ServoTorretta_getAngle; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
215	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.

6.9 Interfaccia semantica della shield motori

File: tank/lib/motorController/motorController.h

Collegamenti audit: A-02, A-11

6.9.1 Panoramica

Questa classe nasconde i registri PCA9685 dietro operazioni leggibili come `DCrun` e `servoTurn`. Le macro mappano porte M1–M4 e S1–S8 a canali PCA. `MAX_SPEED=4096` rappresenta anche il bit full-on/full-off del protocollo PCA, non un normale valore PWM fra 0 e 4095.

6.9.2 Blocchi logici

Righe	Blocco	Cosa studiare
1–48	Identificatori e mappa hardware	Definisce motori, canali di direzione/velocità, direzioni, range PWM e connettori servo.
50–84	Classe e API	Dichiara costruttore, operazioni DC/servo e helper privati verso il driver basso livello.

6.9.3 Funzioni e failure mode

Il costruttore prende frequenza e indirizzo. Le API `DCrun` e `DCbrake` esprimono il controllo motore. Le API `servoTurn` e `servoPairTurn` esprimono un impulso servo. Gli helper privati decidono se usare PWM normale o flag full-on/full-off.

6.9.4 Listing completo numerato

```

1  #ifndef MOTOR_CONTROLLER_H
2  #define MOTOR_CONTROLLER_H
3
4  #include "PWMController.h"
5
6  /*
7   * MOTOR CONTROLLER DELLA SHIELD
8   *
9   * Questo wrapper rende piu' semplice usare la shield:
10  * - comandi motore DC: M1, M2, M3, M4
11  * - comandi servo: S1-S8
12  *
13  * Internamente la shield usa una PCA9685. Ogni uscita motore ha un canale per
14  * direzione e un canale per velocità PWM.
15  */
16
17  // Uscite motore della shield. Uno stepper bipolare dei cingoli usa due uscite.
18  #define M1 1
19  #define M2 2
20  #define M3 3
21  #define M4 4
22
23  // Canali PCA9685 collegati internamente ai driver motore TB6612.
24  #define M1_DIRECTION_CHANNEL 10
25  #define M1_SPEED_CHANNEL 8
26  #define M2_DIRECTION_CHANNEL 11
27  #define M2_SPEED_CHANNEL 13
28  #define M3_DIRECTION_CHANNEL 4
29  #define M3_SPEED_CHANNEL 2
30  #define M4_DIRECTION_CHANNEL 5
31  #define M4_SPEED_CHANNEL 7
32
33  // Direzioni accettate da DCrun().
34  #define FORWARD 1
35  #define BACKWARD 2
36
37  // La PCA9685 ha risoluzione PWM a 12 bit: 0-4096.
38  #define MAX_SPEED 4096
39

```

```

40 // Canali dei connettori servo verificati sulla shield usata in questo progetto.
41 #define S1 0
42 #define S2 1
43 #define S3 3
44 #define S4 6
45 #define S5 9
46 #define S6 12
47 #define S7 14
48 #define S8 15
49
50 class MotorController {
51 public:
52     // La frequenza deve restare a 50 Hz perche' la stessa PCA9685 pilota i servo.
53     explicit MotorController(int frequency, uint8_t address = 0x60);
54
55     // Seleziona l'indirizzo I2C della PCA9685 prima di begin().
56     void setAddress(uint8_t address);
57
58     // Avvia la PCA9685 e applica la frequenza configurata.
59     // false segnala un errore I2C: il chiamante deve entrare in fault sicuro.
60     bool begin();
61
62     // Ritorna false dopo un errore I2C rilevato dal driver PCA9685.
63     bool isCommunicationHealthy() const;
64
65     // Pilota una uscita motore della shield con direzione e velocita PWM.
66     void DCRun(int motorNumber, int direction, int speed);
67
68     // Ferma una o tutte le uscite motore della shield.
69     void DCbrake(int motorNumber);
70     void DCbrakeAll();
71     void bipolarStepperStop(int motorA, int motorB);
72
73     // Manda un comando servo standard 0-180 gradi a un canale servo.
74     void servoTurn(int servoNumber, int degree);
75
76     // Comanda due servo collegati, con il secondo specchiato attorno a un angolo base.
77     void servoPairTurn(int servoA, int servoB, int degree, int invertedDegreeBase);
78
79 private:
80     int frequency;
81     PWMController pwm;
82     int lastDirection[5] = {0, 0, 0, 0, 0};
83
84     // Helper interni: uno per segnali digitali, uno per valori PWM veri.
85     void setDigitalChannel(int channel, bool value);
86     void setPwmChannel(int channel, int value);
87 };
88
89 #endif

```

6.9.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Include il modulo locale PWMController.h e rende visibili la sua interfaccia e i suoi tipi.
18	Definisce la costante di compilazione M1 con valore 1.
19	Definisce la costante di compilazione M2 con valore 2.
20	Definisce la costante di compilazione M3 con valore 3.
21	Definisce la costante di compilazione M4 con valore 4.
24	Definisce la costante di compilazione M1_DIRECTION_CHANNEL con valore 10.
25	Definisce una costante di PWM/frequenza/velocita' per M1_SPEED_CHANNEL.
26	Definisce la costante di compilazione M2_DIRECTION_CHANNEL con valore 11.
27	Definisce una costante di PWM/frequenza/velocita' per M2_SPEED_CHANNEL.
28	Definisce la costante di compilazione M3_DIRECTION_CHANNEL con valore 4.

Riga	Lettura riga per riga
29	Definisce una costante di PWM/frequenza/velocita' per M3_SPEED_CHANNEL.
30	Definisce la costante di compilazione M4_DIRECTION_CHANNEL con valore 5.
31	Definisce una costante di PWM/frequenza/velocita' per M4_SPEED_CHANNEL.
34	Definisce la costante di compilazione FORWARD con valore 1.
35	Definisce la costante di compilazione BACKWARD con valore 2.
38	Definisce una costante di PWM/frequenza/velocita' per MAX_SPEED.
41	Definisce la costante di compilazione S1 con valore 0.
42	Definisce la costante di compilazione S2 con valore 1.
43	Definisce la costante di compilazione S3 con valore 3.
44	Definisce la costante di compilazione S4 con valore 6.
45	Definisce la costante di compilazione S5 con valore 9.
46	Definisce la costante di compilazione S6 con valore 12.
47	Definisce la costante di compilazione S7 con valore 14.
48	Definisce la costante di compilazione S8 con valore 15.
50	Dichiara la classe MotorController e separa l'interfaccia pubblica dai dettagli privati.
51	Cambia la visibilita' dei membri della classe per il compilatore.
53	Dichiara l'API MotorController senza implementarla: la definizione e' in un file .cpp o piu' avanti.
56	Dichiara l'API setAddress senza implementarla: la definizione e' in un file .cpp o piu' avanti.
59	Dichiara l'API begin senza implementarla: la definizione e' in un file .cpp o piu' avanti.
62	Dichiara l'API DCrun senza implementarla: la definizione e' in un file .cpp o piu' avanti.
65	Dichiara l'API DCbrake senza implementarla: la definizione e' in un file .cpp o piu' avanti.
66	Dichiara l'API DCbrakeAll senza implementarla: la definizione e' in un file .cpp o piu' avanti.
67	Dichiara l'API bipolarStepperStop senza implementarla: la definizione e' in un file .cpp o piu' avanti.
70	Dichiara l'API servoTurn senza implementarla: la definizione e' in un file .cpp o piu' avanti.
73	Dichiara l'API servoPairTurn senza implementarla: la definizione e' in un file .cpp o piu' avanti.
75	Cambia la visibilita' dei membri della classe per il compilatore.
76	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
77	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
80	Dichiara l'API setDigitalChannel senza implementarla: la definizione e' in un file .cpp o piu' avanti.
81	Dichiara l'API setPwmChannel senza implementarla: la definizione e' in un file .cpp o piu' avanti.
84	Chiude la include guard o il blocco di precompilazione aperto in precedenza.

6.10 Traduzione motori e servo in canali PCA

File: tank/lib/motorController/motorController.cpp

Collegamenti audit: A-02, A-11

6.10.1 Panoramica

L'implementazione seleziona i due canali corretti per ogni motore, limita il PWM e scrive prima velocità poi direzione. Per i servo converte gradi in tick PWM a 50 Hz. Tutte le operazioni finiscono nel `PWMController`, quindi la loro affidabilità dipende dal bus I2C.

6.10.2 Blocchi logici

Righe	Blocco	Cosa studiare
3-22	Limiti servo e costruzione	Definisce tick servo 110-500, conserva frequenza e permette di cambiare indirizzo prima del begin.
24-54	DCrun	Seleziona la coppia di canali con switch, limita velocità e invia due scritture PCA.
56-90	Stop motori	Spegne PWM e direzione per un motore o per tutti; il nome brake non sostituisce una verifica del comportamento H-bridge.
92-108	Servo	Clampa angoli e calcola l'angolo specchiato della seconda uscita.
110-129	Encoding PCA	Usa 4096 per i flag digitali e tre rami per off, full-on o PWM normale.

6.10.3 Funzioni e failure mode

DCrun rifiuta un numero motore non riconosciuto. DCbrakeAll è usato al boot. servoPairTurn impone un accoppiamento geometrico ma non conosce la meccanica reale. Non vengono restituiti errori I2C: perciò lo strato chiamante non sa se l'uscita ha davvero cambiato stato.

6.10.4 Listing completo numerato

```

1  #include "motorController.h"
2
3  namespace {
4      // Limiti sicuri del pulse servo sulla PCA9685 a 50 Hz.
5      // Evitiamo impulsi troppo estremi per non mandare i servo oltre il fine corsa.
6      const int SERVO_MIN_PULSE = 110;
7      const int SERVO_MAX_PULSE = 500;
8  } // namespace
9
10 MotorController::MotorController(int frequency, uint8_t address)
11     : frequency(frequency), pwm(address) {}
12
13 void MotorController::setAddress(uint8_t address) {
14     // Ricrea il driver PWM con l'indirizzo I2C trovato dal main.
15     pwm = PWMController(address);
16 }
17
18 bool MotorController::begin() {
19     // Avvia il chip PCA9685 e imposta la frequenza condivisa da tutti i canali.
20     return pwm.begin() && pwm.setPWMFreq(frequency);
21 }
22
23 bool MotorController::isCommunicationHealthy() const {
24     return pwm.isCommunicationHealthy();
25 }
26
27 void MotorController::DCrun(int motorNumber, int direction, int speed) {
28     int directionChannel = -1;
29     int speedChannel = -1;

```



```

30
31 // Ogni M della shield corrisponde a due canali PCA9685:
32 // uno decide la direzione, l'altro decide la velocita PWM.
33 switch (motorNumber) {
34 case M1:
35     directionChannel = M1_DIRECTION_CHANNEL;
36     speedChannel = M1_SPEED_CHANNEL;
37     break;
38 case M2:
39     directionChannel = M2_DIRECTION_CHANNEL;
40     speedChannel = M2_SPEED_CHANNEL;
41     break;
42 case M3:
43     directionChannel = M3_DIRECTION_CHANNEL;
44     speedChannel = M3_SPEED_CHANNEL;
45     break;
46 case M4:
47     directionChannel = M4_DIRECTION_CHANNEL;
48     speedChannel = M4_SPEED_CHANNEL;
49     break;
50 default:
51     return;
52 }
53
54 int requestedDirection = direction == FORWARD ? FORWARD : BACKWARD;
55 int requestedSpeed = constrain(speed, 0, MAX_SPEED);
56
57 // Prima di cambiare verso toglie PWM: evita di commutare la direzione
58 // mentre il ponte H e' ancora comandato a velocita diversa da zero.
59 if (lastDirection[motorNumber] != 0 && lastDirection[motorNumber] != requestedDirection) {
60     setPwmChannel(speedChannel, 0);
61 }
62
63 // Il verso viene scritto prima della velocita, cosi' una partenza da fermo
64 // non riceve un impulso con la direzione precedente.
65 setDigitalChannel(directionChannel, requestedDirection == FORWARD);
66 setPwmChannel(speedChannel, requestedSpeed);
67 lastDirection[motorNumber] = requestedDirection;
68 }
69
70 void MotorController::DCbrake(int motorNumber) {
71     // Spegne velocita PWM e porta bassa la direzione del motore scelto.
72     switch (motorNumber) {
73     case M1:
74         setPwmChannel(M1_SPEED_CHANNEL, 0);
75         setDigitalChannel(M1_DIRECTION_CHANNEL, false);
76         lastDirection[M1] = 0;
77         break;
78     case M2:
79         setPwmChannel(M2_SPEED_CHANNEL, 0);
80         setDigitalChannel(M2_DIRECTION_CHANNEL, false);
81         lastDirection[M2] = 0;
82         break;
83     case M3:
84         setPwmChannel(M3_SPEED_CHANNEL, 0);
85         setDigitalChannel(M3_DIRECTION_CHANNEL, false);
86         lastDirection[M3] = 0;
87         break;
88     case M4:
89         setPwmChannel(M4_SPEED_CHANNEL, 0);
90         setDigitalChannel(M4_DIRECTION_CHANNEL, false);
91         lastDirection[M4] = 0;
92         break;
93     }
94 }
95
96 void MotorController::DCbrakeAll() {
97     // Usato all'avvio e nei casi di sicurezza.
98     DCbrake(M1);
99     DCbrake(M2);
100    DCbrake(M3);
101    DCbrake(M4);
102 }
103
104 void MotorController::bipolarStepperStop(int motorA, int motorB) {
105     // Uno stepper bipolare usa due uscite motore: spegniamo entrambe.
106     DCbrake(motorA);
107     DCbrake(motorB);
108 }

```

```

109
110 void MotorController::servoTurn(int servoNumber, int degree) {
111     // Converte i gradi del servo nel valore PWM PCA9685.
112     int constrainedDegree = constrain(degree, 0, 180);
113     int pulse = map(constrainedDegree, 0, 180, SERVO_MIN_PULSE, SERVO_MAX_PULSE);
114     setPwmChannel(servoNumber, pulse);
115 }
116
117 void MotorController::servoPairTurn(int servoA, int servoB, int degree, int invertedDegreeBase) {
118     // Il primo servo segue l'angolo normale.
119     int firstAngle = constrain(degree, 0, 180);
120
121     // Il secondo servo si muove al contrario rispetto al primo.
122     int secondAngle = constrain(invertedDegreeBase - firstAngle, 0, 180);
123
124     servoTurn(servoA, firstAngle);
125     servoTurn(servoB, secondAngle);
126 }
127
128 void MotorController::setDigitalChannel(int channel, bool value) {
129     // Per forzare LOW sulla PCA9685 serve il bit full-off.
130     // Per forzare HIGH serve il bit full-on.
131     pwm.setPWM(channel, value ? 4096 : 0, value ? 0 : 4096);
132 }
133
134 void MotorController::setPwmChannel(int channel, int value) {
135     // value <= 0 spegne completamente il canale.
136     if (value <= 0) {
137         pwm.setPWM(channel, 0, 4096);
138     }
139     // value >= 4096 accende completamente il canale.
140     else if (value >= MAX_SPEED) {
141         pwm.setPWM(channel, 4096, 0);
142     }
143     // Altrimenti genera PWM normale da 0 fino al valore richiesto.
144     else {
145         pwm.setPWM(channel, 0, value);
146     }
147 }

```

6.10.5 Lettura riga per riga

Riga	Letture riga per riga
1	Include il modulo locale motorController.h e rende visibili la sua interfaccia e i suoi tipi.
3	Apri un namespace anonimo: costanti e helper restano privati a questo file di implementazione.
6	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
7	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
10	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
11	Riga sintatticamente significativa del blocco corrente; il suo ruolo è dettagliato dal contesto e dal listing adiacente.
13	Definisce la funzione MotorController::setAddress; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
15	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
18	Definisce la funzione MotorController::begin; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
20	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
21	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
24	Definisce la funzione MotorController::DCrun; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.

Riga	Lettura riga per riga
25	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
26	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
30	Apri una selezione multi-ramo basata sul valore dell'espressione.
31	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
32	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
33	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
34	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
35	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
36	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
37	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
38	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
39	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
40	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
41	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
42	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
43	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
44	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
45	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
46	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
47	Definisce il ramo di fallback dello switch per input non riconosciuti.
48	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
52	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
53	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
56	Definisce la funzione MotorController::DCbrake; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
58	Apri una selezione multi-ramo basata sul valore dell'espressione.
59	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
60	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
61	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
62	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
63	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
64	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.

Riga	Lettura riga per riga
65	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
66	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
67	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
68	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
69	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
70	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
71	Seleziona questo ramo dello switch quando il valore confrontato coincide con il caso indicato.
72	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
73	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
74	Esce dal ramo corrente dello switch evitando il passaggio al caso successivo.
78	Definisce la funzione MotorController::DCbrakeAll; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
80	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
81	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
82	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
83	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
86	Definisce la funzione MotorController::bipolarStepperStop; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
88	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
89	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
92	Definisce la funzione MotorController::servoTurn; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
94	Limita un valore all'intervallo ammesso prima dell'uso successivo.
95	Rimappa linearmente un intervallo intero in un altro; il risultato usa aritmetica intera e puo' troncarsi.
96	Compone un comando verso un canale PCA9685 o verso il relativo helper basso livello.
99	Definisce la funzione MotorController::servoPairTurn; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
101	Limita un valore all'intervallo ammesso prima dell'uso successivo.
104	Limita un valore all'intervallo ammesso prima dell'uso successivo.
106	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
107	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
110	Definisce la funzione MotorController::setDigitalChannel; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
113	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
116	Definisce la funzione MotorController::setPwmChannel; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
118	Valuta una guardia condizionale prima di eseguire il blocco successivo.
119	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
122	Prosegue la catena condizionale: questo ramo e' valutato solo se quelli precedenti non sono stati scelti.
123	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
126	Apri l'alternativa della condizione precedente.
127	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.

6.11 Interfaccia minima PCA9685

File: tank/lib/PWMController/PWMController.h

Collegamenti audit: A-02, A-10

6.11.1 Panoramica

Il driver basso livello conosce indirizzo I2C e registri PCA9685, ma non conosce motori o servo. La separazione consente di riutilizzare l'accesso a registri, ma impone agli strati superiori di controllare errori e validità dei canali.

6.11.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-14	Guardia e responsabilit�	Definisce lo scopo limitato: inizializzazione I2C, frequenza e registri PWM.
15-41	Classe	Espone begin, frequenza e setPWM; conserva registri e indirizzo come dettagli privati.

6.11.3 Funzioni e failure mode

`begin` inizializza il bus e i mode register. `setPWMFreq` programma il prescaler comune. `setPWM` scrive quattro byte di un canale. `readRegister` e `writeRegister` sono primitive I2C private.

6.11.4 Listing completo numerato

```

1  #ifndef PWM_CONTROLLER_H
2  #define PWM_CONTROLLER_H
3
4  #include <Arduino.h>
5
6  /*
7   * DRIVER MINIMALE PCA9685
8   *
9   * La PCA9685 e' il chip PWM della shield. Questo file contiene solo le
10  * operazioni basse livello:
11  * - inizializzazione I2C
12  * - impostazione frequenza PWM
13  * - scrittura dei registri ON/OFF di un canale
14  */
15  class PWMController {
16  public:
17      explicit PWMController(uint8_t address = 0x40);
18
19      // Avvia I2C e resetta i registri base della PCA9685.
20      bool begin();
21
22      // Imposta la frequenza PWM condivisa da tutti i canali.
23      bool setPWMFreq(float frequency);
24
25      // Imposta il tempo ON/OFF di un canale PWM (0-15).
26      bool setPWM(uint8_t channel, uint16_t on, uint16_t off);
27
28      // Resta false dopo un NACK, una lettura corta o un canale non valido.
29      // Il chiamante puo' trasformare questa condizione in un fault sicuro.
30      bool isCommunicationHealthy() const;
31
32  private:
33      // Registri principali della PCA9685.
34      static const uint8_t MODE1_REGISTER = 0x00;
35      static const uint8_t MODE2_REGISTER = 0x01;
36      static const uint8_t PRESCALE_REGISTER = 0xFE;
37      static const uint8_t LED0_ON_L_REGISTER = 0x06;
38
39      // Indirizzo I2C reale della PCA9685.
40      uint8_t address;
41      bool communicationHealthy = true;

```

```

42 // Lettura/scrittura registro via Wire.
43 bool readRegister(uint8_t registerAddress, uint8_t& value);
44 bool writeRegister(uint8_t registerAddress, uint8_t value);
45 };
46
47
48 #endif

```

6.11.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
15	Dichiara la classe PWMController e separa l'interfaccia pubblica dai dettagli privati.
16	Cambia la visibilita' dei membri della classe per il compilatore.
17	Dichiara l'API PWMController senza implementarla: la definizione e' in un file .cpp o piu' avanti.
20	Dichiara l'API begin senza implementarla: la definizione e' in un file .cpp o piu' avanti.
23	Dichiara l'API setPWMFreq senza implementarla: la definizione e' in un file .cpp o piu' avanti.
26	Dichiara l'API setPWM senza implementarla: la definizione e' in un file .cpp o piu' avanti.
28	Cambia la visibilita' dei membri della classe per il compilatore.
30	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
31	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
32	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
33	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
36	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
39	Dichiara l'API readRegister senza implementarla: la definizione e' in un file .cpp o piu' avanti.
40	Dichiara l'API writeRegister senza implementarla: la definizione e' in un file .cpp o piu' avanti.
43	Chiude la include guard o il blocco di precompilazione aperto in precedenza.

6.12 Scritture I2C verso PCA9685

File: tank/lib/PWMController/PWMController.cpp

Collegamenti audit: A-02

6.12.1 Panoramica

Questo e' il punto piu' vicino alla periferica PWM. A 50 Hz il prescaler calcolato e' circa 124. Ogni canale richiede registro base piu' quattro byte ON/OFF. Il codice non controlla il risultato delle transazioni, quindi un errore I2C non raggiunge il controller dei cingoli.

6.12.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-17	Costruzione e begin	Memorizza indirizzo, avvia Wire e programma MODE1/MODE2.
19-36	Frequenza	Corregge il clock, calcola/arrotonda prescaler, entra in sleep, ripristina mode e aspetta 5 ms.
38-48	Scrittura PWM	Indirizza il primo registro del canale e invia i quattro byte in little-endian logico.
50-67	Lettura e scrittura registro	Esegue transazioni minimali; in caso di assenza dati la lettura restituisce zero.

6.12.3 Funzioni e failure mode

`setPWMFreq` usa la formula del datasheet. `setPWM` assume che il canale sia 0-15 e che la transazione vada a buon fine. `readRegister` interpreta nessun byte come zero, valore che puo' confondersi con un registro realmente zero. `writeRegister` non propaga l'esito di `Wire.endTransmission`.

6.12.4 Listing completo numerato

```

1  #include "PWMController.h"
2
3  #include <Wire.h>
4
5  PWMController::PWMController(uint8_t address) : address(address) {}
6
7  bool PWMController::begin() {
8      // Avvia il bus I2C usato per parlare con la PCA9685.
9      Wire.begin();
10     communicationHealthy = true;
11
12     // MODE1 a 0 resetta il funzionamento base.
13     if (!writeRegister(MODE1_REGISTER, 0x00)) {
14         return false;
15     }
16
17     // MODE2 = 0x04 abilita uscite totem-pole/push-pull:
18     // il segnale HIGH e' guidato davvero e non resta flottante.
19     return writeRegister(MODE2_REGISTER, 0x04);
20 }
21
22 bool PWMController::setPWMFreq(float frequency) {
23     if (frequency <= 0.0f) {
24         communicationHealthy = false;
25         return false;
26     }
27
28     // Per cambiare frequenza la PCA9685 deve entrare in sleep mode.
29     float correctedFrequency = frequency * 0.98f;
30
31     // Formula dal datasheet: prescale = clock / (4096 * freq) - 1.
32     float prescaleValue = (25000000.0f / (4096.0f * correctedFrequency)) - 1.0f;
33     uint8_t prescale = static_cast<uint8_t>(floor(prescaleValue + 0.5f));
34

```

```

35     uint8_t oldMode = 0;
36     if (!readRegister(MODE1_REGISTER, oldMode)) {
37         return false;
38     }
39     uint8_t sleepMode = (oldMode & 0x7F) | 0x10;
40
41     // Entra in sleep, scrive prescaler, ripristina modo precedente e riavvia.
42     if (!writeRegister(MODE1_REGISTER, sleepMode) || !writeRegister(PRESCALE_REGISTER, prescale) ||
43         !writeRegister(MODE1_REGISTER, oldMode)) {
44         return false;
45     }
46     delay(5);
47     return writeRegister(MODE1_REGISTER, oldMode | 0xA1);
48 }
49
50 bool PWMController::setPWM(uint8_t channel, uint16_t on, uint16_t off) {
51     if (channel > 15) {
52         communicationHealthy = false;
53         return false;
54     }
55
56     // Ogni canale PCA9685 ha 4 registri consecutivi:
57     // ON_L, ON_H, OFF_L, OFF_H.
58     Wire.beginTransmission(address);
59     Wire.write(LED0_ON_L_REGISTER + (4 * channel));
60     Wire.write(on & 0xFF);
61     Wire.write(on >> 8);
62     Wire.write(off & 0xFF);
63     Wire.write(off >> 8);
64     if (Wire.endTransmission() != 0) {
65         communicationHealthy = false;
66         return false;
67     }
68
69     return true;
70 }
71
72 bool PWMController::readRegister(uint8_t registerAddress, uint8_t& value) {
73     // Prima seleziona il registro da leggere.
74     Wire.beginTransmission(address);
75     Wire.write(registerAddress);
76     if (Wire.endTransmission() != 0) {
77         communicationHealthy = false;
78         return false;
79     }
80
81     // Poi richiede un byte dalla PCA9685.
82     if (Wire.requestFrom(address, static_cast<uint8_t>(1)) != 1 || !Wire.available()) {
83         communicationHealthy = false;
84         return false;
85     }
86
87     value = Wire.read();
88     return true;
89 }
90
91 bool PWMController::writeRegister(uint8_t registerAddress, uint8_t value) {
92     // Scrittura base di un registro PCA9685.
93     Wire.beginTransmission(address);
94     Wire.write(registerAddress);
95     Wire.write(value);
96     if (Wire.endTransmission() != 0) {
97         communicationHealthy = false;
98         return false;
99     }
100
101     return true;
102 }
103
104 bool PWMController::isCommunicationHealthy() const {
105     return communicationHealthy;
106 }

```

6.12.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Include il modulo locale PWMController.h e rende visibili la sua interfaccia e i suoi tipi.
3	Importa l'API I2C Wire usata per parlare con la PCA9685.
5	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
7	Definisce la funzione PWMController::begin; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
9	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
12	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
16	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
19	Definisce la funzione PWMController::setPWMFreq; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
21	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
24	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
25	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
27	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
28	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
31	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
32	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
33	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
34	Blocca intenzionalmente il loop per il tempo indicato; non e' un timer concorrente.
35	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
38	Definisce la funzione PWMController::setPWM; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
41	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
42	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
43	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
44	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
45	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
46	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
47	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
50	Definisce la funzione PWMController::readRegister; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
52	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
53	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
54	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
57	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.

Riga	Lettura riga per riga
58	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
61	Definisce la funzione PWMController::writeRegister; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
63	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
64	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
65	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.
66	Esegue un'operazione I2C sul bus condiviso; il suo esito e la sua durata influenzano il loop cooperativo.

6.13 Configurazione PlatformIO del controller

File: controller-esp32/platformio.ini

Collegamenti audit: A-10

6.13.1 Panoramica

Seleziona un ESP32 Dev Module generico, framework Arduino, monitor a 115200 baud e velocità di upload. `monitor_speed` coincide con `Serial.begin(115200)`. Come il tank, non fissa una versione della platform: il risultato futuro può dipendere dalla risoluzione delle dipendenze.

6.13.2 Blocchi logici

Righe	Blocco	Cosa studiare
2-7	Ambiente ESP32	Indica board, platform, framework, seriale e upload.

6.13.3 Funzioni e failure mode

Non contiene funzioni. E' letto da PlatformIO e determina toolchain e API disponibili al firmware.

6.13.4 Listing completo numerato

```

1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = arduino
5 monitor_speed = 115200
6 upload_speed = 921600

```

6.13.5 Lettura riga per riga

Riga	Lettura riga per riga
2	Riga sintatticamente significativa del blocco corrente; il suo ruolo e' dettagliato dal contesto e dal listing adiacente.
3	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
4	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
5	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
6	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
7	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.

6.14 Scheduler del controller ESP32

File: controller-esp32/src/main.cpp

Collegamenti audit: A-07, A-09, A-11

6.14.1 Panoramica

Il controller usa un loop cooperativo minimo: ogni almeno 10 ms acquisisce input e prova a inviare un datagramma. Non attende il completamento WiFi nel setup. Il periodo e' un limite minimo, non una garanzia esatta: ADC, rete e seriale possono aggiungere tempo.

6.14.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-20	Include, intervallo e stato	Importa i moduli, definisce 10 ms e conserva il timestamp dell'ultimo invio.
22-30	setup	Avvia seriale, calibra/configura ingressi e avvia la procedura WiFi.
32-42	loop	Ritorna prima dello scadere del periodo, poi legge ControllerData e invia UDP.

6.14.3 Funzioni e failure mode

`setup` crea le condizioni iniziali. `loop` e' uno scheduler a polling: non usa task, timer hardware o ISR del progetto. La sottrazione unsigned con `millis` resta corretta al rollover, ma non recupera gli intervalli persi.

6.14.4 Listing completo numerato

```

1  #include <Arduino.h>
2
3  #include "joystickReader.h"
4  #include "udpSender.h"
5
6  /*
7   * MAIN DEL CONTROLLER ESP32
8   *
9   * Metodo di funzionamento:
10  * 1. Configura joystick/pulsanti.
11  * 2. Si collega alla rete WiFi creata dal tank.
12  * 3. Ogni 10 ms legge gli ingressi e invia un pacchetto UDP al tank.
13  * 4. Se perde il WiFi, continua il loop e tenta automaticamente la riconnessione.
14  */
15
16  // 10 ms = circa 100 pacchetti al secondo.
17  // Abbassare troppo questo valore puo' intasare il WiFi senza migliorare molto.
18  #define UDP_SEND_INTERVAL_MS 10
19
20  static unsigned long lastUdpSendTime = 0;
21
22  void setup() {
23      Serial.begin(115200);
24
25      // Configura gli ingressi prima di collegarsi alla rete del tank.
26      JoystickReader_begin();
27      UdpSender_begin();
28
29      Serial.println("Controller pronto");
30  }
31
32  void loop() {
33      unsigned long now = millis();
34      if (now - lastUdpSendTime < UDP_SEND_INTERVAL_MS) {
35          return;
36      }
37      lastUdpSendTime = now;

```

```

38
39 // Legge joystick/pulsanti e invia un comando UDP ogni 10 ms.
40 ControllerData data = JoystickReader_read();
41 UdpSender_send(data);
42 }

```

6.14.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
3	Include il modulo locale joystickReader.h e rende visibili la sua interfaccia e i suoi tipi.
4	Include il modulo locale udpSender.h e rende visibili la sua interfaccia e i suoi tipi.
18	Definisce una durata nominale in millisecondi per UDP_SEND_INTERVAL_MS; viene rispettata solo quando il loop continua a girare.
20	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
22	Definisce la funzione setup; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
23	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
26	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
27	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
29	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
32	Definisce la funzione loop; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
33	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
34	Valuta una guardia condizionale prima di eseguire il blocco successivo.
35	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
37	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
40	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
41	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.

6.15 Contratto dei dati del controller

File: controller-esp32/src/joystickReader.h

Collegamenti audit: A-05, A-07, A-12, A-14

6.15.1 Panoramica

ControllerData e' il contratto fra lettura hardware e trasporto UDP. I quattro assi restano `int` in scala 0-1023; i pulsanti sono booleani. La struct evita che il sender debba conoscere i pin o la calibrazione.

6.15.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-4	Guardia e Arduino	Protegge l'header e rende disponibili tipi/API Arduino.
14-27	ControllerData	Raggruppa assi e pulsanti con nomi coerenti con il protocollo V1.
29-37	API	Dichiara inizializzazione, riarmo neutro dopo reconnect e lettura completa di un campione.

6.15.3 Funzioni e failure mode

JoystickReader_begin configura hardware e calibra. JoystickReader_requireNeutralBeforeCommands riporta il modulo nello stato non armato dopo boot/reconnect. JoystickReader_read restituisce sempre una nuova struct, pronta per la serializzazione oppure sicura quando calibrazione o riarmo non sono validi.

6.15.4 Listing completo numerato

```

1  #ifndef JOYSTICK_READER_H
2  #define JOYSTICK_READER_H
3
4  #include <Arduino.h>
5
6  /*
7   * LETTURA JOYSTICK ESP32
8   *
9   * Questo modulo legge i due joystick analogici e i due pulsanti.
10  * I valori analogici dell'ESP32 vanno da 0 a 4095, ma vengono convertiti
11  * in 0-1023 per usare la stessa scala del firmware tank.
12  */
13
14  // Ingressi letti dall'ESP32 e spediti al tank via UDP.
15  struct ControllerData {
16      // Joy1: guida del tank.
17      int driveX;
18      int driveY;
19
20      // Joy2: torretta ed elevazione.
21      int turretX;
22      int elevationY;
23
24      // Pulsanti digitali.
25      bool zeroPressed;
26      bool firePressed;
27  };
28
29  // Configura i due pulsanti.
30  void JoystickReader_begin();
31
32  // Dopo una riconnessione o una condizione anomala, inibisce i comandi finche'
33  // joystick e pulsanti non restano neutrali/rilasciati per il tempo previsto.
34  void JoystickReader_requireNeutralBeforeCommands();
35
36  // Legge joystick e pulsanti. Gli ADC vengono mappati da 0-4095 a 0-1023.
37  ControllerData JoystickReader_read();
38

```

39 `#endif`

6.15.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
15	Dichiara la struct ControllerData: un contenitore di campi che viaggiano insieme fra moduli.
17	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
18	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
21	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
22	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
25	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
26	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
30	Dichiara JoystickReader_begin(), che configura pulsanti/ADC, calibra e richiede neutralita' iniziale.
34	Dichiara JoystickReader_requireNeutralBeforeCommands(), invocata anche dal sender quando la connessione UDP viene ristabilita.
37	Dichiara JoystickReader_read(), che restituisce dati normalizzati oppure il campione sicuro.
39	Chiude la include guard o il blocco di precompilazione aperto in precedenza.

6.16 ADC, calibrazione e pulsanti ESP32

File: controller-esp32/src/joystickReader.cpp
Collegamenti audit: A-05,A-07,A-12,A-14

6.16.1 Panoramica

Il file legge quattro ingressi ADC1, adatta i 12 bit ESP32 alla scala 0–1023 e corregge il centro dei joystick. GPIO32–35 appartengono ad ADC1 e sono quindi adatti a WiFi sul classico ESP32; GPIO25/26 usano INPUT_PULLUP, per cui LOW significa pulsante premuto. La guida Joy1 applica DRIVE_INPUT_DEADZONE=20; Joy2 mantiene TURRET_INPUT_DEADZONE=80. Sono filtri di rumore all’ingresso, non la protezione motore del tank. La calibrazione blocca intenzionalmente l’avvio per almeno circa 1,24 s: 600 ms iniziali piu’ quattro serie da 80 campioni con delay da 2 ms.

6.16.2 Blocchi logici

Righe	Blocco	Cosa studiare
15–76	Pin, scala, filtri e stati	Definisce ingressi, centro, range, deadzone separate, swap/inversioni e stato di calibrazione/riarmo.
78–86	Stato pulsanti	Dichiara il debounce con candidato, valore stabile e timestamp per Zero e Fire.
88–141	ADC, calibrazione asse e ricentraggio	Legge ADC, rimappa, valida centro/spread e corregge ciascun semiasse con la deadzone ricevuta come argomento.
143–151	Inversione asse	Limita e inverte un asse normalizzato, preservando esattamente il centro.
153–185	Procedura calibrazione	Mostra messaggi, aspetta 600 ms, calibra i quattro assi e restituisce un solo esito booleano.
187–219	Debounce e campione sicuro	Inizializza/legge pulsanti, costruisce dati sicuri e verifica i quattro assi neutrali.
221–238	Begin e riarmo	Configura pull-up/ADC, calibra e invalida l’abilitazione dei comandi.
240–297	Lettura completa	Acquisisce assi con deadzone corretta, gestisce riarmo, swap/inversioni e restituisce il frame locale.

6.16.3 Funzioni e failure mode

readRawJoystick converte 0–4095 in 0–1023. calibrateAxisCenter calcola una media sicura: $80 \times 1023 = 81840$, entro long, e rifiuta assi troppo lontani dal centro o instabili. readCalibratedJoystick ricostruisce i due semiasse rispetto al centro misurato; riceve la deadzone come parametro, così Joy1 usa 20 e Joy2 80 senza duplicare la funzione. invertAxisIfNeeded usa 1023-valore, preservando esplicitamente 512. calibrateJoysticks blocca intenzionalmente il boot. readDebounceButton, safeControllerData e axesAreNeutral sostengono il riarmo. JoystickReader_read realizza il mapping finale: driveX=GPIO32, driveY=GPIO34, turretX=GPIO33, elevationY=GPIO35; le inversioni di Joy1 vengono applicate qui e non sul tank.

6.16.4 Listing completo numerato

```
1 #include "joystickReader.h"
2
3 /*
4  * METODO DI LETTURA DEL CONTROLLER
5  *
6  * L'ESP32 legge i quattro assi analogici con analogRead().
7  * Ogni lettura viene scalata da 0-4095 a 0-1023, così il tank riceve
8  * valori semplici con centro intorno a 512.
9  */
```



```

10  * Importante: i joystick reali quasi mai stanno esattamente al centro.
11  * Per questo il controller calibra il centro all'avvio e applica una
12  * deadzone prima di mandare i dati UDP.
13  */
14
15  // Primo joystick: movimento del tank e sterzata.
16  #define DRIVE_JOYSTICK_X_PIN 34
17  #define DRIVE_JOYSTICK_Y_PIN 32
18
19  // Secondo joystick: torretta orizzontale ed elevazione.
20  #define TURRET_JOYSTICK_X_PIN 35
21  #define TURRET_JOYSTICK_Y_PIN 33
22
23  // I pulsanti usano INPUT_PULLUP, quindi premuto significa LOW.
24  #define ZERO_BUTTON_PIN 25
25  #define FIRE_BUTTON_PIN 26
26
27  // Scala usata dal protocollo UDP del tank.
28  #define JOYSTICK_CENTER 512
29  #define AXIS_MIN 0
30  #define AXIS_MAX 1023
31
32  // Queste deadzone filtrano il rumore ADC attorno al centro fisico calibrato.
33  // Non sono la protezione dei motori: quella resta sul tank, dopo il mixing dei cingoli.
34  // Per Joy1 guida usiamo un valore piccolo, altrimenti si sommerebbe troppo alla
35  // TRACK_COMMAND_DEADZONE del tank e renderebbe poco sensibile la partenza.
36  // 20 e' coerente con la tolleranza di riarmo del ricevitore UDP.
37  #define DRIVE_INPUT_DEADZONE 20
38
39  // Joy2 mantiene la sensibilita' precedente, indipendente dalla guida dei cingoli.
40  #define TURRET_INPUT_DEADZONE 80
41
42  // Numero di letture usate all'avvio per trovare il centro reale dei joystick.
43  #define CALIBRATION_SAMPLES 80
44
45  // La calibrazione viene accettata solo con joystick vicino al centro atteso e
46  // senza variazioni ampie. I valori vanno verificati sul controller fisico.
47  #define CALIBRATION_MAX_CENTER_OFFSET 70
48  #define CALIBRATION_MAX_SPREAD 40
49
50  // Prima di abilitare i comandi, joystick e pulsanti devono restare neutri.
51  #define NEUTRAL_ARMING_MS 400
52
53  // Il pulsante deve mantenere lo stesso livello per questo tempo prima che il
54  // suo stato venga considerato valido.
55  #define BUTTON_DEBOUNCE_MS 40
56
57  // I joystick sono montati ruotati: scambia fisicamente X con Y su entrambi.
58  #define DRIVE_SWAP_X_Y true
59  #define TURRET_SWAP_X_Y true
60
61  // Usa queste quattro costanti solo per cambiare il verso di un singolo asse.
62  // true = 0 diventa 1023 e 1023 diventa 0, mantenendo il centro a 512.
63  // DRIVE_X/Y_INVERTED sono l'unica sorgente di verita' per l'orientamento di Joy1:
64  // il tank riceve gia' assi normalizzati e corregge soltanto il cablaggio dei motori.
65  #define DRIVE_X_INVERTED false
66  #define DRIVE_Y_INVERTED false
67  #define TURRET_X_INVERTED false
68  #define ELEVATION_Y_INVERTED false
69
70  static int driveXCenter = JOYSTICK_CENTER;
71  static int driveYCenter = JOYSTICK_CENTER;
72  static int turretXCenter = JOYSTICK_CENTER;
73  static int elevationYCenter = JOYSTICK_CENTER;
74  static bool calibrationValid = false;
75  static bool commandsArmed = false;
76  static unsigned long neutralSince = 0;
77
78  struct DebouncedButton {
79     uint8_t pin;
80     bool candidatePressed;
81     bool stablePressed;
82     unsigned long candidateChangedAt;
83 };
84
85  static DebouncedButton zeroButton = {ZERO_BUTTON_PIN, false, false, 0};
86  static DebouncedButton fireButton = {FIRE_BUTTON_PIN, false, false, 0};
87
88  static int readRawJoystick(int pin) {

```

```

89     int rawValue = analogRead(pin);
90
91     // L'ADC ESP32 legge 0-4095; il firmware tank lavora in 0-1023.
92     return constrain(map(rawValue, 0, 4095, AXIS_MIN, AXIS_MAX), AXIS_MIN, AXIS_MAX);
93 }
94
95 static bool calibrateAxisCenter(int pin, int& center) {
96     long sum = 0;
97     int minValue = AXIS_MAX;
98     int maxValue = AXIS_MIN;
99
100    for (int i = 0; i < CALIBRATION_SAMPLES; i++) {
101        int value = readRawJoystick(pin);
102        sum += value;
103        minValue = min(minValue, value);
104        maxValue = max(maxValue, value);
105        delay(2);
106    }
107
108    center = constrain(sum / CALIBRATION_SAMPLES, AXIS_MIN, AXIS_MAX);
109
110    bool closeToExpectedCenter =
111        abs(center - JOYSTICK_CENTER) <= CALIBRATION_MAX_CENTER_OFFSET;
112    bool stableDuringSampling = maxValue - minValue <= CALIBRATION_MAX_SPREAD;
113    return closeToExpectedCenter && stableDuringSampling;
114 }
115
116 // Legge, ricentra e normalizza un asse. inputDeadzone e' espressa nella scala UDP 0--1023
117 // ed e' limitata alla corsa disponibile, cosi' map() non riceve un intervallo degenerato.
118 static int readCalibratedJoystick(int pin, int center, int inputDeadzone) {
119     int raw = readRawJoystick(pin);
120     int availableBelowCenter = max(center - AXIS_MIN - 1, 0);
121     int availableAboveCenter = max(AXIS_MAX - center - 1, 0);
122     int deadzone = constrain(inputDeadzone, 0, min(availableBelowCenter, availableAboveCenter));
123
124     // Zona morta attorno al centro reale misurato all'avvio.
125     if (abs(raw - center) < deadzone) {
126         return JOYSTICK_CENTER;
127     }
128
129     // Sotto il centro: usa tutta la corsa 0 -> centro per arrivare a 0 -> 512.
130     // Sopra il centro: usa tutta la corsa centro -> 1023 per arrivare a 512 -> 1023.
131     // Così' correggiamo il centro senza perdere gli estremi del joystick.
132     if (raw < center) {
133         int lowerCenter = max(center - deadzone, 1);
134         return constrain(map(raw, AXIS_MIN, lowerCenter, AXIS_MIN, JOYSTICK_CENTER), AXIS_MIN,
135             JOYSTICK_CENTER);
136     }
137
138     int upperCenter = min(center + deadzone, AXIS_MAX - 1);
139     return constrain(map(raw, upperCenter, AXIS_MAX, JOYSTICK_CENTER, AXIS_MAX), JOYSTICK_CENTER,
140         AXIS_MAX);
141 }
142
143 static int invertAxisIfNeeded(int value, bool inverted) {
144     value = constrain(value, AXIS_MIN, AXIS_MAX);
145
146     if (!inverted || value == JOYSTICK_CENTER) {
147         return value;
148     }
149
150     return AXIS_MAX - value;
151 }
152
153 static bool calibrateJoysticks() {
154     Serial.println();
155     Serial.println("=== CALIBRAZIONE JOYSTICK ESP32 ===");
156     Serial.println("Lascia Joy1 e Joy2 fermi al centro...");
157     delay(600);
158
159     bool driveXValid = calibrateAxisCenter(DRIVE_JOYSTICK_X_PIN, driveXCenter);
160     bool driveYValid = calibrateAxisCenter(DRIVE_JOYSTICK_Y_PIN, driveYCenter);
161     bool turretXValid = calibrateAxisCenter(TURRET_JOYSTICK_X_PIN, turretXCenter);
162     bool elevationYValid = calibrateAxisCenter(TURRET_JOYSTICK_Y_PIN, elevationYCenter);
163
164     Serial.print("Joy1 guida centro: X=");
165     Serial.print(driveXCenter);
166     Serial.print(" Y=");
167     Serial.print(driveYCenter);

```

```

168     Serial.println();
169
170     Serial.print("Joy2 torretta centro: X=");
171     Serial.print(turretXCenter);
172     Serial.print(" Y=");
173     Serial.println(elevationYCenter);
174     Serial.println("Da ora un asse fermo viene mandato come circa 512.");
175     Serial.println("=====");
176     Serial.println();
177
178     bool valid = driveXValid && driveYValid && turretXValid && elevationYValid;
179     if (!valid) {
180         Serial.println("ERRORE: joystick non centrato o instabile.");
181         Serial.println("Comandi inibiti: riavvia lasciando i joystick fermi al centro.");
182     }
183
184     return valid;
185 }
186
187 static void beginDebounceButton(DebounceButton& button) {
188     bool pressed = digitalRead(button.pin) == LOW;
189     button.candidatePressed = pressed;
190     button.stablePressed = pressed;
191     button.candidateChangedAt = millis();
192 }
193
194 static bool readDebounceButton(DebounceButton& button) {
195     bool rawPressed = digitalRead(button.pin) == LOW;
196     unsigned long now = millis();
197
198     if (rawPressed != button.candidatePressed) {
199         button.candidatePressed = rawPressed;
200         button.candidateChangedAt = now;
201     }
202
203     if (button.stablePressed != button.candidatePressed &&
204         now - button.candidateChangedAt >= BUTTON_DEBOUNCE_MS) {
205         button.stablePressed = button.candidatePressed;
206     }
207
208     return button.stablePressed;
209 }
210
211 static ControllerData safeControllerData() {
212     return {JOYSTICK_CENTER, JOYSTICK_CENTER, JOYSTICK_CENTER, JOYSTICK_CENTER, false, false};
213 }
214
215 static bool axesAreNeutral(int drivePhysicalX, int drivePhysicalY, int turretPhysicalX,
216                             int turretPhysicalY) {
217     return drivePhysicalX == JOYSTICK_CENTER && drivePhysicalY == JOYSTICK_CENTER &&
218         turretPhysicalX == JOYSTICK_CENTER && turretPhysicalY == JOYSTICK_CENTER;
219 }
220
221 void JoystickReader_begin() {
222     // INPUT_PULLUP evita resistenze esterne: il pin sta HIGH e va LOW quando premi.
223     pinMode(ZERO_BUTTON_PIN, INPUT_PULLUP);
224     pinMode(FIRE_BUTTON_PIN, INPUT_PULLUP);
225
226     analogReadResolution(12);
227     analogSetAttenuation(ADC_11db);
228
229     beginDebounceButton(zeroButton);
230     beginDebounceButton(fireButton);
231     calibrationValid = calibrateJoysticks();
232     JoystickReader_requireNeutralBeforeCommands();
233 }
234
235 void JoystickReader_requireNeutralBeforeCommands() {
236     commandsArmed = false;
237     neutralSince = 0;
238 }
239
240 ControllerData JoystickReader_read() {
241     int drivePhysicalX =
242         readCalibratedJoystick(DRIVE_JOYSTICK_X_PIN, driveXCenter, DRIVE_INPUT_DEADZONE);
243     int drivePhysicalY =
244         readCalibratedJoystick(DRIVE_JOYSTICK_Y_PIN, driveYCenter, DRIVE_INPUT_DEADZONE);
245     int turretPhysicalX =
246         readCalibratedJoystick(TURRET_JOYSTICK_X_PIN, turretXCenter, TURRET_INPUT_DEADZONE);

```

```

247 int turretPhysicalY =
248     readCalibratedJoystick(TURRET_JOYSTICK_Y_PIN, elevationYCenter, TURRET_INPUT_DEADZONE);
249 bool zeroPressed = readDebounceButton(zeroButton);
250 bool firePressed = readDebounceButton(fireButton);
251
252 if (!calibrationValid) {
253     return safeControllerData();
254 }
255
256 if (!commandsArmed) {
257     bool neutralAndReleased =
258         axesAreNeutral(drivePhysicalX, drivePhysicalY, turretPhysicalX, turretPhysicalY) &&
259         !zeroPressed && !firePressed;
260
261     if (!neutralAndReleased) {
262         neutralSince = 0;
263         return safeControllerData();
264     }
265
266     unsigned long now = millis();
267     if (neutralSince == 0) {
268         neutralSince = now;
269         return safeControllerData();
270     }
271
272     if (now - neutralSince < NEUTRAL_ARMING_MS) {
273         return safeControllerData();
274     }
275
276     commandsArmed = true;
277     Serial.println("Comandi joystick armati.");
278 }
279
280 ControllerData data;
281
282 // Joy1 viene usato dal tank per il mixing differenziale dei cingoli.
283 int driveX = DRIVE_SWAP_X_Y ? drivePhysicalY : drivePhysicalX;
284 int driveY = DRIVE_SWAP_X_Y ? drivePhysicalX : drivePhysicalY;
285 data.driveX = invertAxisIfNeeded(driveX, DRIVE_X_INVERTED);
286 data.driveY = invertAxisIfNeeded(driveY, DRIVE_Y_INVERTED);
287
288 // Joy2 viene usato dal tank per torretta orizzontale ed elevazione.
289 int turretX = TURRET_SWAP_X_Y ? turretPhysicalY : turretPhysicalX;
290 int elevationY = TURRET_SWAP_X_Y ? turretPhysicalX : turretPhysicalY;
291 data.turretX = invertAxisIfNeeded(turretX, TURRET_X_INVERTED);
292 data.elevationY = invertAxisIfNeeded(elevationY, ELEVATION_Y_INVERTED);
293
294 data.zeroPressed = zeroPressed;
295 data.firePressed = firePressed;
296
297 return data;
298 }

```

6.16.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Include il modulo locale joystickReader.h e rende visibili la sua interfaccia e i suoi tipi.
16	Configura il pin hardware tramite la macro DRIVE_JOYSTICK_X_PIN; il valore associato deve coincidere con il cablaggio.
17	Configura il pin hardware tramite la macro DRIVE_JOYSTICK_Y_PIN; il valore associato deve coincidere con il cablaggio.
20	Configura il pin hardware tramite la macro TURRET_JOYSTICK_X_PIN; il valore associato deve coincidere con il cablaggio.
21	Configura il pin hardware tramite la macro TURRET_JOYSTICK_Y_PIN; il valore associato deve coincidere con il cablaggio.
24	Configura il pin hardware tramite la macro ZERO_BUTTON_PIN; il valore associato deve coincidere con il cablaggio.

Riga	Lettura riga per riga
25	Configura il pin hardware tramite la macro FIRE_BUTTON_PIN; il valore associato deve coincidere con il cablaggio.
28	Definisce una soglia di neutralita' del joystick per JOYSTICK_CENTER.
29	Definisce la costante di compilazione AXIS_MIN con valore 0.
30	Definisce la costante di compilazione AXIS_MAX con valore 1023.
33	Definisce una soglia di neutralita' del joystick per CONTROLLER_DEADZONE.
36	Definisce la costante di compilazione CALIBRATION_SAMPLES con valore 80.
39	Configura il verso o lo scambio logico dell'asse DRIVE_SWAP_X_Y.
40	Configura il verso o lo scambio logico dell'asse TURRET_SWAP_X_Y.
44	Configura il verso o lo scambio logico dell'asse DRIVE_X_INVERTED.
45	Configura il verso o lo scambio logico dell'asse DRIVE_Y_INVERTED.
46	Configura il verso o lo scambio logico dell'asse TURRET_X_INVERTED.
47	Configura il verso o lo scambio logico dell'asse ELEVATION_Y_INVERTED.
49	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
50	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
51	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
52	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
54	Definisce la funzione readRawJoystick; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
55	Acquisisce un campione ADC; il risultato dipende da tensione, attenuazione, rumore e cablaggio.
58	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
61	Definisce la funzione calibrateAxisCenter; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
62	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
64	Avvia un ciclo con inizializzazione, condizione di prosecuzione e avanzamento espliciti.
65	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
66	Blocca intenzionalmente il loop per il tempo indicato; non e' un timer concorrente.
69	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
72	Definisce la funzione readCalibratedJoystick; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
73	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
76	Valuta una guardia condizionale prima di eseguire il blocco successivo.
77	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
83	Valuta una guardia condizionale prima di eseguire il blocco successivo.
84	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
85	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
86	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
89	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
90	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
91	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
94	Definisce la funzione invertAxisIfNeeded; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
95	Limita un valore all'intervallo ammesso prima dell'uso successivo.

Riga	Lettura riga per riga
97	Valuta una guardia condizionale prima di eseguire il blocco successivo.
98	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
101	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
104	Definisce la funzione <code>calibrateJoysticks</code> ; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
105	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
106	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
107	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
108	Blocca intenzionalmente il loop per il tempo indicato; non e' un timer concorrente.
110	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
111	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
112	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
113	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
115	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
116	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
117	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
118	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
119	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
121	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
122	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
123	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
124	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
125	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
126	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
127	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
130	Definisce la funzione <code>JoystickReader_begin</code> ; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
132	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
133	Configura elettricamente la direzione del GPIO prima di leggerlo o pilotarlo.
135	Configura il convertitore ADC ESP32 prima della lettura degli assi.
136	Configura il convertitore ADC ESP32 prima della lettura degli assi.
138	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
141	Definisce la funzione <code>JoystickReader_read</code> ; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.

Riga	Lettura riga per riga
142	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
144	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
145	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
146	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
147	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
150	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
151	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
152	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
153	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
156	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
157	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
158	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
159	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
162	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
163	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
165	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.

6.17 Interfaccia del sender UDP

File: controller-esp32/src/udpSender.h

Collegamenti audit: A-04, A-09, A-10

6.17.1 Panoramica

L'header dichiara il modulo che collega `ControllerData` a WiFi/UDP. Il tipo di ritorno `void` di `UdpSender_send` indica che il chiamante non riceve una conferma di consegna dal tank.

6.17.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-6	Guardia e dipendenze	Include Arduino e il contratto dati del joystick.
16-22	API	Dichiara avvio rete e invio del frame V1.

6.17.3 Funzioni e failure mode

`UdpSender_begin` avvia la connessione station. `UdpSender_send` tenta l'invio solo quando WiFi risulta associato, ma l'associazione non e' conferma di ricezione UDP dal tank.

6.17.4 Listing completo numerato

```

1  #ifndef UDP_SENDER_H
2  #define UDP_SENDER_H
3
4  #include <Arduino.h>
5
6  #include "joystickReader.h"
7
8  /*
9   * INVIO UDP ESP32
10  *
11  * Questo modulo prende i dati letti da joystickReader e li manda al tank.
12  * Il controller non controlla direttamente i motori: spedisce solo coordinate
13  * e pulsanti usando il protocollo V1.
14  */
15
16 // Avvia la connessione al tank. Se il WiFi cade, il modulo ritenta senza bloccare il loop.
17 void UdpSender_begin();
18
19 // Invia un comando ControllerData usando il protocollo UDP V1.
20 void UdpSender_send(const ControllerData& data);
21
22 #endif

```

6.17.5 Lettura riga per riga

Riga	Letture riga per riga
1	Inizia una include guard: il corpo dell'header verra' elaborato solo se la macro non e' gia' definita.
2	Definisce la macro della include guard: impedisce inclusioni multiple dell'header.
4	Importa il core Arduino: tipi, GPIO, tempo, map e constrain.
6	Include il modulo locale joystickReader.h e rende visibili la sua interfaccia e i suoi tipi.
17	Dichiara l'API <code>UdpSender_begin</code> senza implementarla: la definizione e' in un file .cpp o piu' avanti.
20	Dichiara l'API <code>UdpSender_send</code> senza implementarla: la definizione e' in un file .cpp o piu' avanti.

Riga	Lettura riga per riga
22	Chiude la include guard o il blocco di precompilazione aperto in precedenza.

6.18 Connessione station e serializzazione UDP

File: controller-esp32/src/udpSender.cpp

Collegamenti audit: A-04, A-05, A-09, A-10

6.18.1 Panoramica

Il controller opera come station della rete creata dal tank. Conserva socket, IP di fallback, timer e flag di transizione. Quando WiFi torna connesso usa il gateway come indirizzo del tank e riapre il socket. Il messaggio V1 e' costruito con `snprintf`; il frame normale massimo V1;1023;1023;1023;1023;1;1 e' lungo 26 byte, quindi entra in `message[64]`.

6.18.2 Blocchi logici

Righe	Blocco	Cosa studiare
1-39	Librerie, parametri e stato	Definisce credenziali, porta, periodi, socket, IP e timer persistenti.
41-58	Configurazione socket	Legge gateway DHCP, applica fallback, chiude/apre UDP e stampa diagnosi.
60-91	Macchina a stati WiFi	Distingue connesso, transizione dopo perdita e tentativo di riconnessione ogni 3 s.
93-102	Begin	Imposta station mode, evita persistenza NVS, abilita auto reconnect e avvia WiFi.begin.
104-144	Invio	Verifica associazione, serializza V1, prova beginPacket, scrive, chiude e stampa una diagnostica periodica.

6.18.3 Funzioni e failure mode

`isEmptyIp` riconosce 0.0.0.0. `configureUdpConnection` aggiorna `tankIP` e il socket.

`ensureWifiConnected` e' una macchina a stati: la prima connessione configura UDP, una perdita ferma il socket e ogni 3 s tenta reconnect/begin.

`UdpSender_send` tratta `beginPacket==0` come fallimento, ma non controlla `udp.begin`, `udp.print` o `udp.endPacket`; anche un invio allo stack locale non e' un ACK dal tank.

6.18.4 Listing completo numerato

```

1 #include "udpSender.h"
2
3 #include <WiFi.h>
4 #include <WiFiUdp.h>
5
6 /*
7  * METODO DI INVIO UDP
8  *
9  * Il tank crea la rete WiFi. L'ESP32 si collega come client e manda pacchetti
10  * UDP all'indirizzo IP del tank. Il messaggio e' una stringa compatta:
11  *
12  * V1;driveX;driveY;turretX;elevationY;zero;fire
13  */
14
15 // Credenziali WiFi esposte dal tank Uno R4.
16 const char* WIFI_SSID = "Tank_AP";
17 const char* WIFI_PASS = "12345678";
18
19 // Deve essere uguale alla porta usata da udpReceiver.cpp sul tank.
20 #define UDP_PORT 4210
21
22 // Stampa coordinate una volta al secondo.
23 #define COORDINATE_PRINT_INTERVAL_MS 1000
24
25 // Se il WiFi cade, prova a ricollegarsi senza bloccare il loop.

```

```

26 #define WIFI_RECONNECT_INTERVAL_MS 3000
27
28 static WiFiUDP udp;
29
30 // IP tipico dell'access point Arduino. Se il gateway WiFi e' disponibile, viene aggiornato.
31 static IPAddress tankIP(192, 168, 4, 1);
32 static unsigned long lastCoordinatePrintTime = 0;
33 static unsigned long lastReconnectAttemptTime = 0;
34 static bool wifiWasConnected = false;
35 static bool udpReady = false;
36 static bool suppressCommandAfterConnection = false;
37
38 // Controlla se un IP e' 0.0.0.0, cioe' non valido.
39 static bool isEmptyIp(IPAddress ip) {
40     return ip[0] == 0 && ip[1] == 0 && ip[2] == 0 && ip[3] == 0;
41 }
42
43 // Aggiorna l'indirizzo del tank e riapre UDP dopo ogni nuova connessione.
44 // Un socket non aperto non viene trattato come canale di controllo operativo.
45 static bool configureUdpConnection() {
46     // Il gateway dell'access point e' l'indirizzo IP del tank.
47     IPAddress gateway = WiFi.gatewayIP();
48     if (!isEmptyIp(gateway)) {
49         tankIP = gateway;
50     }
51
52     udp.stop();
53     udpReady = udp.begin(UDP_PORT) == 1;
54     lastCoordinatePrintTime = millis();
55     // Il pacchetto costruito prima della riconnessione potrebbe contenere un
56     // pulsante premuto. Impone di nuovo neutralita' prima di trasmettere dati.
57     JoystickReader_requireNeutralBeforeCommands();
58     suppressCommandAfterConnection = true;
59
60     if (!udpReady) {
61         Serial.println("ERRORE: socket UDP controller non disponibile");
62         return false;
63     }
64
65     Serial.println("WiFi collegato al tank");
66     Serial.print("Controller IP: ");
67     Serial.println(WiFi.localIP());
68     Serial.print("Tank IP: ");
69     Serial.println(tankIP);
70     return true;
71 }
72
73 // Ritorna true quando il controller puo' inviare dati al tank.
74 // In caso di perdita WiFi, tenta la riconnessione ogni 3 secondi senza delay().
75 static bool ensureWifiConnected() {
76     unsigned long now = millis();
77
78     if (WiFi.status() == WL_CONNECTED) {
79         if (!wifiWasConnected) {
80             wifiWasConnected = true;
81             lastReconnectAttemptTime = now;
82             return configureUdpConnection();
83         }
84
85         // Anche con WiFi associato, il socket puo' non essere stato aperto.
86         if (!udpReady && now - lastReconnectAttemptTime >= WIFI_RECONNECT_INTERVAL_MS) {
87             lastReconnectAttemptTime = now;
88             return configureUdpConnection();
89         }
90
91         return udpReady;
92     }
93
94     if (wifiWasConnected) {
95         wifiWasConnected = false;
96         udpReady = false;
97         udp.stop();
98         Serial.println("WiFi perso: il tank entrera' in timeout di sicurezza");
99     }
100
101     if (now - lastReconnectAttemptTime >= WIFI_RECONNECT_INTERVAL_MS) {
102         lastReconnectAttemptTime = now;
103         Serial.println("Tentativo di riconnessione a Tank_AP...");
104     }

```

```

105     // reconnect() riusa SSID e password impostati da WiFi.begin().
106     // Se non parte, begin() ricrea esplicitamente la connessione station.
107     if (!WiFi.reconnect()) {
108         WiFi.begin(WIFI_SSID, WIFI_PASS);
109     }
110 }
111
112 return false;
113 }
114
115 void UdpSender_begin() {
116     Serial.print("Connessione a ");
117     Serial.println(WIFI_SSID);
118
119     WiFi.mode(WIFI_STA);
120     WiFi.persistent(false);
121     WiFi.setAutoReconnect(true);
122     WiFi.begin(WIFI_SSID, WIFI_PASS);
123     udpReady = false;
124     lastReconnectAttemptTime = millis();
125 }
126
127 void UdpSender_send(const ControllerData& data) {
128     if (!ensureWifiConnected()) {
129         // Il tank fermerà i movimenti dopo il proprio timeout di 500 ms.
130         return;
131     }
132
133     // Scarta il campione letto prima del cambio stato WiFi. Dal ciclo
134     // successivo JoystickReader_read() restituisce solo il comando sicuro
135     // finché joystick e pulsanti non sono neutrali e rilasciati.
136     if (suppressCommandAfterConnection) {
137         suppressCommandAfterConnection = false;
138         return;
139     }
140
141     char message[64];
142
143     // Protocollo finale: V1;driveX;driveY;turretX;elevationY;zero;fire
144     int messageLength =
145         snprintf(message, sizeof(message), "V1;%d;%d;%d;%d;%d", data.driveX, data.driveY,
146             data.turretX, data.elevationY, data.zeroPressed ? 1 : 0, data.firePressed ? 1 : 0);
147
148     if (messageLength <= 0 || messageLength >= static_cast<int>(sizeof(message))) {
149         Serial.println("ERRORE: pacchetto UDP controller non valido");
150         return;
151     }
152
153     if (udp.beginPacket(tankIP, UDP_PORT) == 0) {
154         // beginPacket() può fallire se la rete non è pronta.
155         udpReady = false;
156         return;
157     }
158
159     if (udp.print(message) != static_cast<size_t>(messageLength) || udp.endPacket() != 1) {
160         udpReady = false;
161         return;
162     }
163
164     // Stampa le coordinate una volta al secondo, non ad ogni pacchetto.
165     if (millis() - lastCoordinatePrintTime > COORDINATE_PRINT_INTERVAL_MS) {
166         lastCoordinatePrintTime = millis();
167
168         Serial.println();
169         Serial.println("=== ESP32 TX UDP ===");
170         Serial.print("Joy1 guida X=");
171         Serial.print(data.driveX);
172         Serial.print(" Y=");
173         Serial.println(data.driveY);
174         Serial.print("Joy2 torretta X=");
175         Serial.print(data.turretX);
176         Serial.print(" elevazione Y=");
177         Serial.println(data.elevationY);
178         Serial.print("Pulsanti zero=");
179         Serial.print(data.zeroPressed ? "PREMUTO" : "off");
180         Serial.print(" fire=");
181         Serial.println(data.firePressed ? "PREMUTO" : "off");
182         Serial.println("=====");
183     }

```

6.18.5 Lettura riga per riga

Riga	Lettura riga per riga
1	Include il modulo locale udpSender.h e rende visibili la sua interfaccia e i suoi tipi.
3	Importa l'API WiFi della board corrente.
4	Importa il socket UDP fornito dal framework.
16	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
17	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
20	Fissa la porta di comunicazione UDP_PORT condivisa dai due firmware.
23	Definisce una durata nominale in millisecondi per COORDINATE_PRINT_INTERVAL_MS; viene rispettata solo quando il loop continua a girare.
26	Definisce una durata nominale in millisecondi per WIFI_RECONNECT_INTERVAL_MS; viene rispettata solo quando il loop continua a girare.
28	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
31	Dichiara l'API tankIP senza implementarla: la definizione e' in un file .cpp o piu' avanti.
32	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
33	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
34	Dichiara stato statico: dura per l'intera esecuzione e, se locale a funzione, conserva valore fra chiamate.
37	Definisce la funzione isEmptyIp; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
38	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
42	Definisce la funzione configureUdpConnection; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
44	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
45	Valuta una guardia condizionale prima di eseguire il blocco successivo.
46	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
49	Invoca il socket UDP; un successo locale non equivale a conferma di ricezione remota.
50	Invoca il socket UDP; un successo locale non equivale a conferma di ricezione remota.
51	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
53	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
54	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
55	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
56	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
57	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.

Riga	Lettura riga per riga
62	Definisce la funzione ensureWifiConnected; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
63	Valuta una guardia condizionale prima di eseguire il blocco successivo.
64	Valuta una guardia condizionale prima di eseguire il blocco successivo.
65	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
66	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
69	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
72	Valuta una guardia condizionale prima di eseguire il blocco successivo.
73	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
74	Invoca il socket UDP; un successo locale non equivale a conferma di ricezione remota.
75	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
78	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
79	Valuta una guardia condizionale prima di eseguire il blocco successivo.
80	Aggiorna o inizializza uno stato; seguire la variabile permette di vedere l'effetto sul ciclo successivo.
81	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
85	Valuta una guardia condizionale prima di eseguire il blocco successivo.
86	Invoca l'API WiFi; associazione, gateway e stato rete sono gestiti dal framework, non dal protocollo applicativo.
90	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
93	Definisce la funzione UdpSender_begin; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
94	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
95	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
97	Invoca l'API WiFi; associazione, gateway e stato rete sono gestiti dal framework, non dal protocollo applicativo.
98	Invoca l'API WiFi; associazione, gateway e stato rete sono gestiti dal framework, non dal protocollo applicativo.
99	Invoca l'API WiFi; associazione, gateway e stato rete sono gestiti dal framework, non dal protocollo applicativo.
100	Invoca l'API WiFi; associazione, gateway e stato rete sono gestiti dal framework, non dal protocollo applicativo.
101	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
104	Definisce la funzione UdpSender_send; i parametri costituiscono il suo input e il corpo seguente ne realizza il contratto.
105	Valuta una guardia condizionale prima di eseguire il blocco successivo.
107	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
110	Dichiara o inizializza una variabile: il tipo determina intervallo, durata e semantica del dato.
113	Serializza testo dentro un buffer con limite esplicito di dimensione.
114	Esegue l'istruzione o la chiamata indicata nel contesto della funzione corrente.
116	Valuta una guardia condizionale prima di eseguire il blocco successivo.
118	Termina la funzione e restituisce al chiamante il valore o il controllo indicato.
121	Invoca il socket UDP; un successo locale non equivale a conferma di ricezione remota.

Riga	Lettura riga per riga
122	Invoca il socket UDP; un successo locale non equivale a conferma di ricezione remota.
125	Valuta una guardia condizionale prima di eseguire il blocco successivo.
126	Legge o confronta il contatore temporale Arduino; la sottrazione unsigned e' rollover-safe ma richiede un loop che torni a eseguire.
128	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
129	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
130	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
131	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
132	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
133	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
134	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
135	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
136	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
137	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
138	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
139	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
140	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
141	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.
142	Emette diagnostica seriale; e' osservabilita' utile ma puo' aggiungere tempo al giro di loop.

7 Riepilogo di studio e limiti pratici

- Il firmware e' modulare: controller legge input, UDP trasporta, tank decide e moduli attuano.
- Buffer e calcoli correnti sono generalmente limitati; i rischi principali non sono un overflow classico, ma fiducia in I/O, rete e stato fisico.
- Non esistono interrupt applicativi: tutti i timer dipendono dal polling regolare.
- Riarmo, debounce, fault AP/I2C e dead-time sono stati rafforzati nel software; il parser V1 e' invece mantenuto compatibile con la baseline e lascia A-08 aperto. Limiti torretta, timeout reale e impulso relay restano da provare elettricamente e meccanicamente.
- Il PDF corrente unisce la baseline e il delta post-correzioni. Dopo il prossimo commit stabile conviene rigenerare completamente anche le tabelle riga per riga dei moduli modificati, per riallineare ogni numero di riga al listing dinamico.